asturnet.

MF0227_3 · IFCD0112

UF2404
Principios de la programación orientada a objetos

Manual del alumno · 90 horas

Unidad formativa UF2404 del módulo MF0227_3 «Programación orientada a objetos» (250 horas).
Certificado de profesionalidad «Programación con lenguajes orientados a objetos y bases de datos relacionales» (RD 628/2013) · Formación telemática en aula virtual

Índice

Presentación de la unidad 6

Tema 1. Introducción al paradigma orientado a objetos 7

El ciclo de desarrollo del software con objetos . . . 7
Modularidad y abstracción . . . 7
Programación estructurada frente a orientada a objetos . . . 8
Conceptos básicos: objeto, clase y mensaje . . . 9
Los cuatro pilares de la programación orientada a objetos . . . 9
Del problema a las clases: cómo encontrar los objetos . . . 9
Calidad del software y métricas orientadas a objetos . . . 10
Actividad en el aula virtual: Modelar una biblioteca . . . 10
Resumen del tema . . . 10
Glosario . . . 11

Tema 2. Clases y objetos 12

La clase: atributos, métodos y encapsulación . . . 12
Anatomía de una clase . . . 12
Visibilidad: quién puede ver qué . . . 14
Objetos: instanciación, estado, comportamiento e identidad . . . 14
Constructores y destrucción de objetos . . . 14
La instancia actual: this y self . . . 15
Tipos de métodos según su función . . . 15
Miembros de instancia y miembros de clase (static) . . . 15
Representación UML de una clase . . . 16
Actividad en el aula virtual: Diseña y codifica una clase . . . 16
Resumen del tema . . . 16
Glosario . . . 17

Tema 3. Generalización y especialización: herencia 18

Generalización y especialización . . . 18
Herencia en el código: extends y super . . . 18
Transmisión de atributos y métodos . . . 19
Sobreescritura de métodos y uso de super . . . 20
La clase raíz . . . 20
Creación de objetos en la herencia . . . 20

Herencia múltiple y sus problemas . . . 21
Cuándo NO usar herencia . . . 22
Actividad en el aula virtual: Jerarquía de empleados . . . 22
Resumen del tema . . . 22
Glosario . . . 22

Tema 4. Relaciones entre clases 23

Por qué importan las relaciones . . . 23
Asociación . . . 23
Agregación . . . 24
Composición . . . 24
Clases contenedoras y objetos colección . . . 25
Diagrama de clases con relaciones . . . 26
Composición frente a herencia . . . 26
Actividad en el aula virtual: Del enunciado al diagrama . . . 26
Resumen del tema . . . 27
Glosario . . . 27

Tema 5. Polimorfismo, clases abstractas e interfaces 28

¿Qué es el polimorfismo? . . . 28
Tipos de polimorfismo . . . 28
Clases abstractas . . . 29
Interfaces . . . 29
Subclases concretas y uso polimórfico . . . 30
Sobrecarga de métodos . . . 31
Conversión de tipos y comprobación en ejecución . . . 31
Diagrama UML: interfaz y realización . . . 31
Actividad en el aula virtual: Añadir una figura sin tocar el bucle . . . 32
Resumen del tema . . . 32
Glosario . . . 32

Tema 6. Técnicas de programación estructurada 33

Programación estructurada dentro de la POO . . . 33
Variables, tipos y expresiones . . . 33
Estructuras de control: selección . . . 34
Estructuras de control: repetición . . . 34
Funciones y procedimientos . . . 35
Paso de parámetros . . . 35

Recursividad . . . 35
Llamadas a métodos dentro de una clase . . . 36
Actividad en el aula virtual: Refactorizar un método largo . . . 36
Resumen del tema . . . 36
Glosario . . . 36

Tema 7. Estructuras de información y gestión de memoria 37

Tipos de datos simples . . . 37
Arrays . . . 37
Colecciones: listas, conjuntos y mapas . . . 38
Código con colecciones . . . 38
Estructuras no lineales: árboles y grafos . . . 38
Implementar una estructura propia . . . 39
Gestión de memoria: pila y montón . . . 40
Referencias, copia e igualdad . . . 40
Recolección de basura y fugas de memoria . . . 40
Actividad en el aula virtual: Elegir la estructura adecuada . . . 41
Resumen del tema . . . 41
Glosario . . . 41

Tema 8. Lenguajes orientados a objetos y librerías de clases 42

Lenguajes orientados a objetos: panorama . . . 42
Criterios para elegir lenguaje . . . 42
Tipos de aplicaciones . . . 43
Herramientas de desarrollo . . . 43
Librerías de clases estándar . . . 43
Cadenas y utilidades básicas . . . 44
Fechas, streams y Optional . . . 44
Cómo leer la documentación de una API . . . 45
Actividad en el aula virtual: Cazadores de clases . . . 45
Resumen del tema . . . 45
Glosario . . . 45

Tema 9. Implementación del paradigma: excepciones, eventos e hilos 47

Del diseño al código: receta para implementar una clase . . . 47
Estructura de un proyecto Java . . . 47
Tipos enumerados y registros . . . 48
Excepciones: concepto y jerarquía . . . 48

Excepciones propias y su tratamiento 49
Gestión de eventos: el patrón observador 50
Eventos en código 51
Hilos: ejecución concurrente 52
Crear y sincronizar hilos 52
Buenas prácticas con hilos 53
Acceso a bases de datos desde las aplicaciones 54
Proyecto integrador de la UF2404 54
Actividad en el aula virtual: Reaccionar a eventos con hilos 54
Resumen del tema 54
Glosario 54

Presentación de la unidad

Este manual acompaña a las sesiones de la unidad formativa **UF2404: Principios de la programación orientada a objetos**, de 90 horas. Cada tema contiene la explicación de los conceptos, ejemplos de código en **Java** con su equivalente en **Python**, actividades para el aula virtual y un resumen. Los ejemplos están en la carpeta `codigo/` y pueden ejecutarse tal cual.

Tema	Título	Horas
1	Introducción al paradigma orientado a objetos	6
2	Clases y objetos	10
3	Generalización y especialización: herencia	10
4	Relaciones entre clases	8
5	Polimorfismo, clases abstractas e interfaces	8
6	Técnicas de programación estructurada	10
7	Estructuras de información y gestión de memoria	10
8	Lenguajes orientados a objetos y librerías de clases	8
9	Implementación del paradigma: excepciones, eventos e hilos	20
	Total	**90**

> **Cómo usar el manual.** Lee cada tema antes de la sesión síncrona, ejecuta los ejemplos y apunta tus dudas. Las prácticas y los cuestionarios se entregan en la plataforma.

Tema 1. Introducción al paradigma orientado a objetos

En este tema. Duración orientativa: 6 horas. Situar la programación orientada a objetos dentro del ciclo de desarrollo del software.

Objetivos de aprendizaje

- Situar la programación orientada a objetos dentro del ciclo de desarrollo del software.
- Distinguir el paradigma estructurado del orientado a objetos.
- Definir con precisión objeto, clase y mensaje, y los cuatro pilares de la POO.
- Identificar clases, atributos y métodos a partir de la descripción de un problema.

Criterios de evaluación del módulo que se trabajan

- CE1.1 Ciclo de desarrollo del software bajo el paradigma OO
- CE1.3 Objeto frente a clase
- CE2.1 Criterios de calidad y métricas OO

El ciclo de desarrollo del software con objetos

Desarrollar software no consiste solo en escribir código. Es un proceso que empieza entendiendo el problema y termina cuando el programa se retira. En el paradigma orientado a objetos (POO) las mismas ideas —los objetos— acompañan al equipo en todas las fases: se descubren durante el análisis, se detallan en el diseño y se convierten en clases en la programación.

Esta continuidad es una de las grandes ventajas del paradigma: el vocabulario del cliente («pedido», «socio», «préstamo») reaparece casi sin traducir en el código, lo que facilita la comunicación y el mantenimiento.

- **Análisis:** qué debe hacer el sistema; se identifican los objetos del dominio.
- **Diseño:** cómo lo hará; clases, responsabilidades, relaciones (UML).
- **Programación orientada a objetos:** las clases se codifican en un lenguaje (Java, C#, Python...).
- **Pruebas y puesta en marcha:** se verifica cada clase y el conjunto.
- **Mantenimiento:** los cambios afectan a pocas clases si el diseño es bueno.

Idea clave. La programación es **una fase** del ciclo de desarrollo, no el ciclo completo.

Modularidad y abstracción

Un programa grande es inmanejable si se escribe como un bloque. La **modularidad** consiste en dividirlo en piezas pequeñas con una responsabilidad clara. La **abstracción** consiste en quedarse solo con lo esencial de un elemento del mundo real para el problema que se resuelve: de un «alumno» nos puede interesar su nombre y sus notas, pero no su color de pelo.

En la POO la unidad de modularidad es la clase: agrupa los datos y las operaciones que trabajan con ellos.

- Modularidad: piezas pequeñas, cohesionadas y poco acopladas.
- Abstracción: modelar solo lo relevante para el problema.
- Ocultación de información: lo interno de la pieza no es visible desde fuera.
- Resultado: código más fácil de entender, probar y reutilizar.

Programación estructurada frente a orientada a objetos

En la programación estructurada el programa se organiza en torno a **procedimientos** que reciben datos y los transforman; los datos suelen estar separados y accesibles desde cualquier parte. En la POO el programa se organiza en torno a **objetos** que combinan datos y comportamiento.

Ninguno de los dos es «mejor» en abstracto: dentro de los métodos de una clase seguimos usando estructuras de control estructuradas (secuencia, decisión, repetición). Lo que cambia es dónde vive cada pieza de código.

El mismo problema, dos paradigmas

Java

```java
// Paradigma estructurado: los datos y los procedimientos van por separado
static double[] saldos = {100.0, 250.0};

static void ingresar(int cuenta, double importe) {
    saldos[cuenta] += importe;
}
```

Python

```python
# Paradigma estructurado: los datos y los procedimientos van por separado
saldos = [100.0, 250.0]


def ingresar(cuenta, importe):
    saldos[cuenta] += importe
```

Versión orientada a objetos

Java

```java
// Paradigma orientado a objetos: datos + comportamiento en una misma unidad
static class Cuenta {
    private double saldo;

    Cuenta(double inicial) { this.saldo = inicial; }

    void ingresar(double importe) { saldo += importe; }

    double getSaldo() { return saldo; }
}
```

Python

```python
# Paradigma orientado a objetos: datos + comportamiento en una misma unidad
class Cuenta:
    def __init__(self, inicial):
        self._saldo = inicial

    def ingresar(self, importe):
        self._saldo += importe
```

```
    @property
    def saldo(self):
        return self._saldo
```

Fíjate. Los datos (`saldo`) y las operaciones (`ingresar`) viajan juntos: quien usa la cuenta no manipula el dato directamente.

Conceptos básicos: objeto, clase y mensaje

Un **objeto** es una entidad con identidad propia, un estado (los valores de sus atributos) y un comportamiento (lo que sabe hacer). Una **clase** es la plantilla a partir de la cual se crean objetos del mismo tipo. Un **mensaje** es la petición que un objeto hace a otro para que ejecute uno de sus métodos: `POO = objetos + mensajes`.

Concepto	Qué es	Ejemplo
Clase	Plantilla: define atributos y métodos	`Cuenta` (titular, saldo, ingresar())
Objeto (instancia)	Elemento concreto creado desde la clase, con identidad y estado propios	La cuenta de Ana con 500 EUR
Atributo	Dato que guarda el estado	`saldo`
Método	Operación que define el comportamiento	`retirar(100)`
Mensaje	Invocación de un método sobre un objeto	`cuentaAna.retirar(100)`

Los cuatro pilares de la programación orientada a objetos

Cuatro ideas sostienen el paradigma. Las veremos con detalle en los temas siguientes; conviene tener ahora el mapa completo.

Pilar	Idea	Beneficio
Abstracción	Representar solo las características esenciales	Modelos manejables
Encapsulación	Ocultar el estado y exponer una interfaz controlada	Datos protegidos; cambios internos sin romper a los clientes
Herencia	Una clase reutiliza y amplía a otra	Reutilización; jerarquías de conceptos
Polimorfismo	Un mismo mensaje, comportamientos distintos según el objeto	Código flexible y extensible

Del problema a las clases: cómo encontrar los objetos

Una técnica sencilla para empezar es analizar la descripción del problema: los **sustantivos** suelen ser candidatos a clases o atributos y los **verbos** a métodos. Después se descartan los que no aportan y se completan responsabilidades.

Ejemplo: «Una biblioteca presta libros a sus socios. Cada préstamo tiene una fecha de devolución». Candidatos: Biblioteca, Libro, Socio, Préstamo (clases); fecha de devolución (atributo de Préstamo); prestar, devolver (métodos).

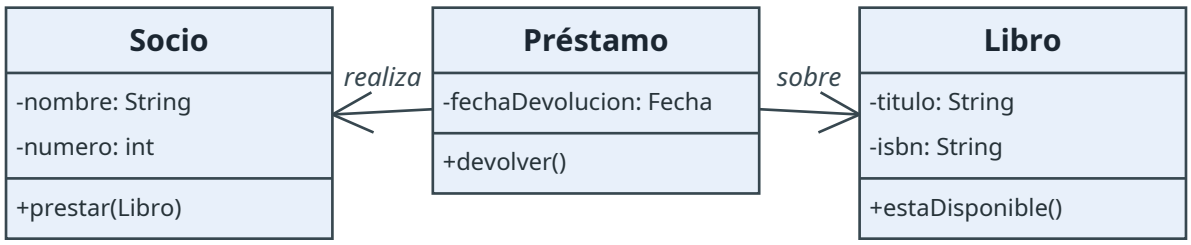


Figura 1.1. Primer diagrama de clases de la biblioteca (notación UML simplificada).

Calidad del software y métricas orientadas a objetos

Un buen diseño orientado a objetos no se mide por el número de clases, sino por atributos de calidad. Estos criterios guiarán todo el curso y se evalúan en la UF2406 con métricas concretas.

Criterio	Significado	Métrica orientada a objetos
Cohesión	Los miembros de una clase hablan de lo mismo	Falta de cohesión de métodos (LCOM)
Acoplamiento	Cuánto depende una clase de otras	Acoplamiento entre objetos (CBO)
Herencia razonable	Jerarquías poco profundas y con sentido	Profundidad del árbol de herencia (DIT)
Complejidad	Facilidad de comprender y probar los métodos	Complejidad ciclomática; métodos ponderados por clase (WMC)
Reutilización / mantenibilidad	Facilidad de usar y modificar	Nº de clases afectadas por un cambio

> **Regla práctica.** Buscamos **alta cohesión y bajo acoplamiento**.

Actividad en el aula virtual: Modelar una biblioteca

Modelar una biblioteca · 20 minutos

En grupos, a partir del enunciado de la biblioteca, identificad clases, atributos y métodos.

- Subrayad sustantivos y verbos del enunciado (5 min).
- Proponed 4-5 clases con 2 atributos y 1 método cada una (10 min).
- Cada grupo expone su propuesta y se comparan decisiones (5 min).

Resumen del tema

- La POO es una fase dentro del ciclo de desarrollo: análisis diseño programación pruebas mantenimiento.
- Objeto = identidad + estado + comportamiento; clase = plantilla de objetos.

- Pilares: abstracción, encapsulación, herencia y polimorfismo.
- Sustantivos clases/atributos; verbos métodos.
- Buscamos alta cohesión y bajo acoplamiento.

Glosario

Término	Significado
Objeto	Entidad con identidad, estado y comportamiento.
Clase	Plantilla que describe atributos y métodos de un tipo de objetos.
Mensaje	Petición enviada a un objeto para que ejecute un método.
Instancia	Objeto creado a partir de una clase.
Abstracción	Seleccionar solo lo esencial del elemento modelado.

Tema 2. Clases y objetos

En este tema. Duración orientativa: 10 horas. Declarar clases con atributos, métodos y constructores en Java y en Python.

Objetivos de aprendizaje

- Declarar clases con atributos, métodos y constructores en Java y en Python.
- Aplicar la encapsulación mediante modificadores de acceso.
- Crear objetos, enviar mensajes y distinguir identidad, estado y comportamiento.
- Usar la referencia a la instancia actual (this / self) y los miembros de clase (static).

Criterios de evaluación del módulo que se trabajan

- CE1.2 Características de una clase: atributos, métodos y encapsulación
- CE1.3-1.4 Objeto frente a clase; estado y comportamiento
- CE2.4 Uso de this / self
- CE2.2 Creación y destrucción de objetos

La clase: atributos, métodos y encapsulación

Una clase describe un tipo de objetos: qué datos guarda (**atributos**) y qué operaciones ofrece (**métodos**). La **encapsulación** es el mecanismo por el que la clase oculta sus atributos y obliga a usar sus métodos para consultarlos o modificarlos, de modo que el objeto siempre se mantiene en un estado válido.

Piensa en un cajero automático: ves la pantalla y el teclado (la interfaz), no el mecanismo interno que cuenta los billetes.

- Atributos: variables que representan el **estado** del objeto.
- Métodos: funciones que definen el **comportamiento** y trabajan sobre el estado.
- Interfaz de la clase: el conjunto de miembros visibles desde fuera.
- Encapsulación: atributos privados + métodos públicos que validan.

Anatomía de una clase

El ejemplo recoge todos los elementos básicos: atributos de instancia y de clase, constructores (uno sobrecargado), métodos de consulta y de modificación y el método `toString` para obtener una representación en texto.

Clase Cuenta

Java

```java
static class Cuenta {
    private static int totalCuentas = 0;     // atributo de clase (compartido)

    private final String titular;            // atributos de instancia: estado
    private double saldo;

    // Constructor
    Cuenta(String titular, double saldoInicial) {
```

```
        this.titular = titular;              // this: instancia actual
        this.saldo = saldoInicial;
        totalCuentas++;
    }

    // Constructor sobrecargado
    Cuenta(String titular) { this(titular, 0); }

    // Método modificador: cambia el estado respetando las reglas del negocio
    void retirar(double importe) {
        if (importe <= 0 || importe > saldo) {
            throw new IllegalArgumentException("Importe no válido: " + importe);
        }
        saldo -= importe;
    }

    void ingresar(double importe) {
        if (importe <= 0) throw new IllegalArgumentException("Importe no válido");
        saldo += importe;
    }

    // Métodos de consulta
    double getSaldo() { return saldo; }
    String getTitular() { return titular; }
    static int getTotalCuentas() { return totalCuentas; }

    @Override
    public String toString() { return titular + ": " + saldo + " EUR"; }
}
```

Python

```
class Cuenta:
    total_cuentas = 0                    # atributo de clase (compartido)

    def __init__(self, titular, saldo_inicial=0):
        self._titular = titular          # atributos de instancia: estado
        self._saldo = saldo_inicial      # el guion bajo indica "uso interno"
        Cuenta.total_cuentas += 1

    def retirar(self, importe):
        if importe <= 0 or importe > self._saldo:
            raise ValueError(f"Importe no válido: {importe}")
        self._saldo -= importe

    def ingresar(self, importe):
        if importe <= 0:
            raise ValueError("Importe no válido")
        self._saldo += importe

    @property
    def saldo(self):
        return self._saldo

    @property
    def titular(self):
        return self._titular

    def __str__(self):
        return f"{self._titular}: {self._saldo} EUR"
```

> **Observa.** Los atributos son privados; las reglas del negocio (importe válido) viven dentro de los métodos.

Visibilidad: quién puede ver qué

Java dispone de cuatro niveles de acceso; Python se basa en convenciones de nombres. La regla general es **exponer lo mínimo**: atributos privados y solo los métodos necesarios como públicos.

Java	Alcance	Python (convención)
`public`	Cualquier clase	`nombre` (sin guion)
`protected`	Mismo paquete y subclases	`_nombre` (uso interno / subclases)
(por defecto)	Solo el mismo paquete	—
`private`	Solo la propia clase	`__nombre` (name mangling)

> **Ojo.** Python no impide el acceso: confía en la convención. Java lo comprueba al compilar.

Objetos: instanciación, estado, comportamiento e identidad

Un objeto se crea con el operador `new` (Java) o llamando a la clase (Python). Cada objeto tiene su propia copia de los atributos de instancia: dos cuentas con el mismo saldo siguen siendo dos objetos distintos. La **identidad** es lo que distingue a un objeto de otro, aunque tengan el mismo estado; por eso `==` compara referencias y `equals` compara contenido.

- Estado: valores actuales de los atributos (cambia con el tiempo).
- Comportamiento: lo que el objeto sabe hacer (sus métodos).
- Identidad: referencia única en memoria; dos objetos iguales no son el mismo objeto.

Crear objetos y enviar mensajes

Java

```java
Cuenta a = new Cuenta("Ana", 500);       // instanciación
Cuenta b = new Cuenta("Luis");
a.retirar(120);
b.ingresar(75.5);
System.out.println(a);
System.out.println(b);
System.out.println("Cuentas creadas: " + Cuenta.getTotalCuentas());
```

Python

```python
a = Cuenta("Ana", 500)
b = Cuenta("Luis")
a.retirar(120)
b.ingresar(75.5)
print(a)
print(b)
print("Cuentas creadas:", Cuenta.total_cuentas)
```

Constructores y destrucción de objetos

El **constructor** es el método especial que inicializa el objeto al crearlo: no tiene tipo de retorno y se llama igual que la clase (`__init__` en Python). Pueden existir varios constructores con distintos parámetros (**sobrecarga**).

La **destrucción** funciona de forma distinta a otros lenguajes como C++: en Java y Python el programador no libera la memoria; lo hace el **recolector de basura** cuando el objeto ya no es alcanzable. Para liberar recursos externos (ficheros, conexiones) se usa `try-with-resources` / `with`, no los finalizadores.

- Constructor: reserva e inicializa el estado; debe dejar el objeto consistente.
- Constructor por defecto: lo genera el compilador solo si no escribes ninguno.
- Destructor: no se invoca de forma determinista en Java/Python.
- Libera recursos externos con `AutoCloseable` / gestores de contexto.

> **Cuidado.** No confundas «dejar de referenciar» un objeto con «destruirlo»: la memoria se libera cuando el recolector decide.

La instancia actual: this y self

Dentro de un método, `this` (Java) o `self` (Python) es la referencia al objeto que recibió el mensaje. Sirve para distinguir un atributo de un parámetro con el mismo nombre, para encadenar constructores (`this(...)`) y para pasar el propio objeto a otros métodos. En Python `self` es un parámetro explícito de cada método; en Java es implícito.

this frente a self

Java

```java
class Punto {
    private int x;
    Punto(int x) { this.x = x; }      // this.x = atributo, x = parámetro
    Punto sumar(int n) { x += n; return this; }   // devuelve la propia instancia
    // (encadenamiento)
}
```

Python

```python
class Punto:
    def __init__(self, x):
        self._x = x                   # self referencia a la instancia actual

    def sumar(self, n):
        self._x += n
        return self                   # permite p.sumar(1).sumar(2)
```

Tipos de métodos según su función

Tipo	Propósito	Ejemplo
Constructor	Inicializar el objeto	`Cuenta(String, double)`
Consulta (getter)	Leer el estado sin modificarlo	`getSaldo()`
Modificador (setter / comando)	Cambiar el estado respetando las reglas	`ingresar(double)`
Utilidad	Cálculos auxiliares o conversión	`toString()`
Destrucción / cierre	Liberar recursos externos	`close()`

Miembros de instancia y miembros de clase (static)

Un atributo **de instancia** existe una vez por objeto. Un atributo **de clase** (`static` en Java, atributo definido en el cuerpo de la clase en Python) existe una sola vez y se comparte entre todos los objetos: sirve para contadores, constantes o configuraciones. Los métodos `static` no usan `this` porque no actúan sobre ningún objeto concreto.

- `static int totalCuentas` un único valor para todas las instancias.
- Se accede con el nombre de la clase: `Cuenta.getTotalCuentas()`.
- Los métodos static no pueden usar atributos de instancia.
- Evita abusar de estado estático mutable: dificulta las pruebas.

Representación UML de una clase

En UML una clase se dibuja como un rectángulo con tres compartimentos: nombre, atributos y métodos. El prefijo `-` indica privado, `+` público, `#` protegido; los miembros estáticos se subrayan.

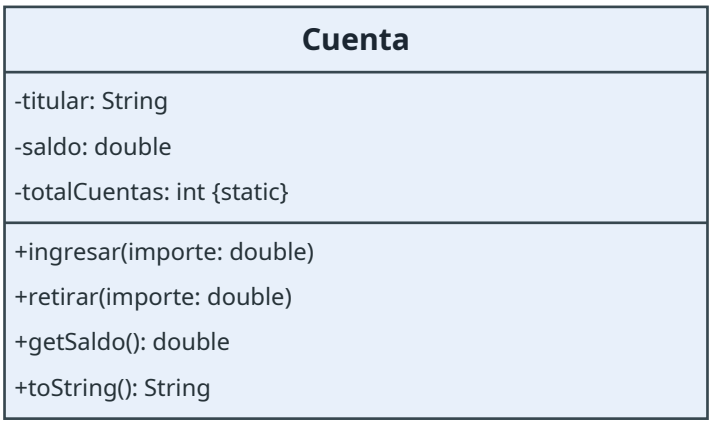


Figura 2.1. Clase Cuenta en UML.

Actividad en el aula virtual: Diseña y codifica una clase

Diseña y codifica una clase · 30 minutos

Programad la clase `Producto` (nombre, precio, stock) con constructor validado y los métodos `vender(n)` y `reponer(n)`.

- Dibujad primero la clase en UML (5 min).
- Implementadla en Java o Python en parejas con pantalla compartida (20 min).
- Probad con un `main` que cubra un caso correcto y uno erróneo (5 min).

Resumen del tema

- Una clase agrupa atributos (estado) y métodos (comportamiento) y oculta su interior mediante encapsulación.
- Los constructores inicializan objetos y pueden sobrecargarse; la memoria la libera el recolector de basura.
- `this` / `self` es la referencia al objeto que recibe el mensaje.
- Los miembros static / de clase se comparten entre todas las instancias.

- Identidad (`==`) no es lo mismo que igualdad de contenido (`equals`).

Glosario

Término	Significado
Encapsulación	Ocultar el estado interno y ofrecer una interfaz controlada.
Constructor	Método especial que inicializa un objeto al crearlo.
Sobrecarga	Varios métodos o constructores con el mismo nombre y distintos parámetros.
Recolector de basura	Mecanismo que libera automáticamente los objetos no alcanzables.
this / self	Referencia a la instancia actual.

Tema 3. Generalización y especialización: herencia

En este tema. Duración orientativa: 10 horas. Explicar la herencia como relación «es un» y sus características.

Objetivos de aprendizaje

- Explicar la herencia como relación «es un» y sus características.
- Construir jerarquías de clases con constructores, sobreescritura y `super`.
- Comprender el papel de la clase raíz (`Object`) y la creación de objetos en una jerarquía.
- Analizar los problemas de la herencia múltiple y cómo se resuelven en Java y Python.

Criterios de evaluación del módulo que se trabajan

- CE1.5 Características de la herencia
- CE1.6 Herencia múltiple y sus problemas
- CE2.4 Uso de super y this
- CE2.10 Construcción de clases con herencia

Generalización y especialización

En el mundo real clasificamos por categorías: un coche **es un** vehículo, un vehículo **es una** máquina. La **generalización** extrae lo común de varias clases en una superclase; la **especialización** crea subclases que añaden o cambian comportamiento. La herencia es el mecanismo del lenguaje que implementa esta idea: la subclase hereda atributos y métodos de la superclase y puede ampliarlos.

Beneficios: reutilización de código y jerarquías que reflejan el dominio. Riesgo: usarla solo para «ahorrar código» cuando no existe una relación «es un» real.

- Superclase (clase base, padre): contiene lo común.
- Subclase (clase derivada, hija): hereda y especializa.
- Prueba del «es un»: *¿un Coche es un Vehículo?* Si no, probablemente no debe heredar.
- Herencia **simple**: una sola superclase. Herencia **múltiple**: varias.

Regla. Hereda para **especializar**, no para reutilizar a toda costa. Si dudas, usa composición.

Herencia en el código: extends y super

La subclase se declara con `extends` en Java y con paréntesis en Python. Hereda todos los miembros no privados. Para inicializar la parte heredada, el constructor de la subclase **debe llamar al de la superclase** (`super(...)`) como primera instrucción en Java.

Superclase Vehiculo

Java

```java
static class Vehiculo {
    protected String matricula;
    protected int velocidad;

    Vehiculo(String matricula) {
        this.matricula = matricula;
        System.out.println("Constructor de Vehiculo");
    }

    void acelerar(int incremento) { velocidad += incremento; }

    String describir() { return "Vehículo " + matricula + " a " + velocidad + " km/h"; }
}
```

Python

```python
class Vehiculo:
    def __init__(self, matricula):
        self.matricula = matricula
        self.velocidad = 0
        print("Constructor de Vehiculo")

    def acelerar(self, incremento):
        self.velocidad += incremento

    def describir(self):
        return f"Vehículo {self.matricula} a {self.velocidad} km/h"
```

Subclase Coche

Java

```java
static class Coche extends Vehiculo {
    private int puertas;

    Coche(String matricula, int puertas) {
        super(matricula);                       // el constructor padre se ejecuta
        // primero
        this.puertas = puertas;
        System.out.println("Constructor de Coche");
    }

    @Override
    String describir() {                        // sobreescritura (override)
        return "Coche de " + puertas + " puertas. " + super.describir();
    }
}
```

Python

```python
class Coche(Vehiculo):
    def __init__(self, matricula, puertas):
        super().__init__(matricula)      # el constructor padre se ejecuta primero
        self.puertas = puertas
        print("Constructor de Coche")

    def describir(self):                 # sobreescritura
        return f"Coche de {self.puertas} puertas. " + super().describir()
```

> **Orden de ejecución.** Al crear un `Coche` se ejecuta primero el constructor de `Vehiculo` y después el de `Coche`.

Transmisión de atributos y métodos

Miembro de la superclase	¿Lo hereda la subclase?	¿Accesible por nombre?
`public`	Sí	Sí
`protected`	Sí	Sí (en la subclase y el paquete)
Sin modificador	Sí si está en el mismo paquete	Solo en el mismo paquete
`private`	Existe en el objeto pero no es accesible	No: se usa mediante métodos de la superclase
Constructores	No se heredan	Se invocan con `super(...)`

Sobreescritura de métodos y uso de super

La subclase puede **sobreescribir** (override) un método heredado para dar una versión especializada. Con `super.metodo()` se reutiliza la versión de la superclase. En Java conviene anotar con `@Override`: el compilador avisa si la firma no coincide. No confundir con **sobrecarga** (mismo nombre, distinta firma, en la misma clase).

- Sobreescritura: misma firma, distinta implementación, en la subclase.
- `super.describir()` invoca la implementación de la superclase.
- Un método `final` no puede sobreescribirse; una clase `final` no puede heredarse.
- La versión que se ejecuta depende del tipo **real** del objeto (enlace dinámico).

La clase raíz

En Java toda clase hereda de `java.lang.Object` (directa o indirectamente); en Python, de `object`. Por eso todos los objetos disponen de métodos como `toString()`, `equals()` y `hashCode()` (`__str__`, `__eq__`, `__hash__` en Python). Sobreescribirlos es una práctica habitual.

Método heredado de Object	Para qué sirve	Cuándo sobreescribirlo
`toString()`	Representación en texto	Casi siempre: facilita depurar y mostrar
`equals(Object)`	Igualdad de contenido	Cuando dos objetos distintos puedan ser «iguales»
`hashCode()`	Código para colecciones hash	Siempre que sobreescribas `equals`
`getClass()`	Tipo real del objeto	No se sobreescribe

Creación de objetos en la herencia

Al crear un objeto de una subclase se construye **toda la cadena**: primero la raíz, luego cada clase intermedia y por último la propia clase. Si la superclase no tiene constructor sin parámetros, la subclase está obligada a llamar explícitamente a uno con `super(...)`.

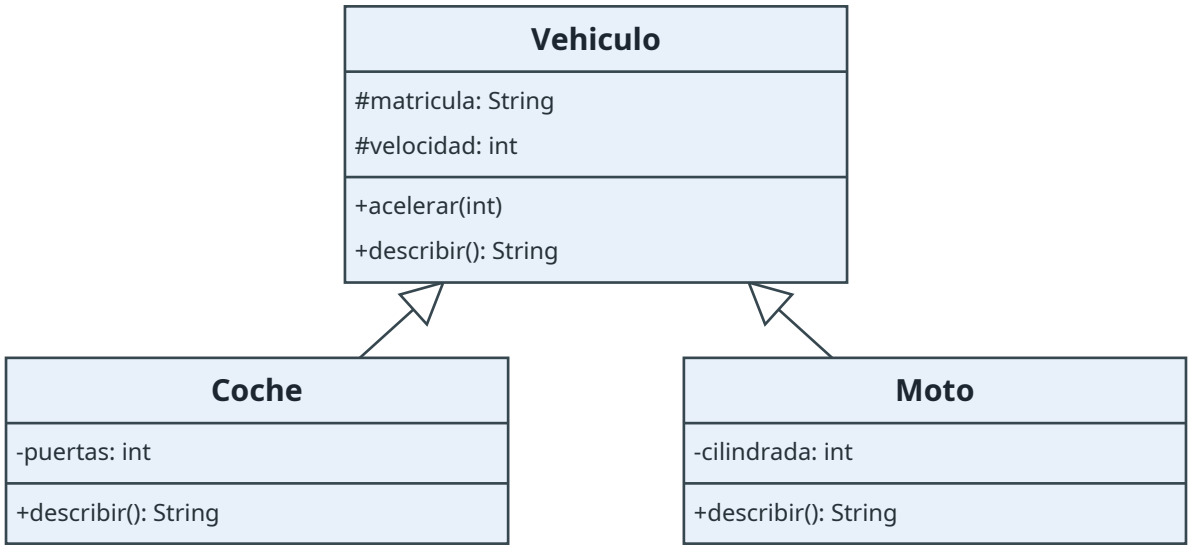


Figura 3.1. Jerarquía de herencia. La flecha de triángulo hueco apunta a la superclase.

Herencia múltiple y sus problemas

En la herencia múltiple una clase hereda de varias superclases. Es potente, pero introduce el **problema del diamante**: si dos superclases heredan a su vez de una misma clase y definen el mismo método, ¿cuál debe usar la subclase? Cada lenguaje lo resuelve a su manera.

Java **no permite** herencia múltiple de clases; la sustituye por la implementación de varias **interfaces** (con métodos `default`). Python sí la permite y usa un orden de resolución de métodos (MRO) determinista.

Herencia múltiple

Java

```java
// Java no tiene herencia múltiple de clases; la simula con interfaces
interface Volador { default String volar() { return "Volando"; } }
interface Nadador { default String nadar() { return "Nadando"; } }

static class Pato implements Volador, Nadador { }
```

Python

```python
# Python sí admite herencia múltiple (orden de resolución: MRO)
class Volador:
    def volar(self):
        return "Volando"


class Nadador:
    def nadar(self):
        return "Nadando"


class Pato(Volador, Nadador):
    pass
```

Conclusión. Si necesitas combinar capacidades, prefiere interfaces o composición antes que jerarquías múltiples.

Cuándo NO usar herencia

- Cuando no se cumple «es un»: un `Pedido` no es una `Lista`, **tiene** una lista de líneas.
- Cuando la jerarquía crece demasiado en profundidad (más de 3-4 niveles) y es difícil de entender.
- Cuando la subclase rechaza parte de lo heredado (viola el principio de sustitución de Liskov).
- Alternativa: **composición** (tema siguiente) e interfaces.

Actividad en el aula virtual: Jerarquía de empleados

Jerarquía de empleados · 30 minutos

Una empresa tiene empleados fijos, temporales y directivos. Diseñad la jerarquía de clases y calculad la nómina.

- Identificad atributos comunes y los específicos (10 min).
- Dibujad el diagrama UML con herencia (5 min).
- Implementad `calcularNomina()` sobreescrito en cada subclase (15 min).

Resumen del tema

- Herencia = relación «es un»: la subclase reutiliza y especializa a la superclase.
- El constructor padre se ejecuta antes; `super(...)` lo invoca.
- Sobreescribir cambia la implementación; el método ejecutado depende del tipo real.
- Toda clase hereda de `Object`/`object`.
- Java usa interfaces en lugar de herencia múltiple de clases; Python usa MRO.

Glosario

Término	Significado
Superclase	Clase de la que se hereda.
Subclase	Clase que hereda de otra.
Override	Redefinir en la subclase un método heredado.
Problema del diamante	Ambigüedad al heredar de dos clases que comparten ancestro.
MRO	Orden de resolución de métodos en Python.

Tema 4. Relaciones entre clases

En este tema. Duración orientativa: 8 horas. Distinguir asociación, agregación, composición, generalización y dependencia.

Objetivos de aprendizaje

- Distinguir asociación, agregación, composición, generalización y dependencia.
- Implementar cada relación con referencias y colecciones.
- Elegir entre herencia y composición.
- Leer y dibujar diagramas de clases con multiplicidades.

Criterios de evaluación del módulo que se trabajan

- CE2.3 Mecanismos de implementación de las relaciones entre clases
- CE2.10 Relaciones de agregación, composición y asociación
- CE1.8 Interpretación de documentación de diseño

Por qué importan las relaciones

Los objetos útiles rara vez trabajan solos: un pedido tiene líneas, un curso tiene un profesor, un alumno se matricula en cursos. Las **relaciones entre clases** expresan cómo se conectan y quién depende de quién. Elegir bien la relación determina el acoplamiento y el ciclo de vida de los objetos.

Relación	Significado	Pregunta que la identifica	Ciclo de vida
Dependencia	Usa temporalmente a otra clase	¿Solo la necesita dentro de un método?	Independiente
Asociación	Conoce a otra clase	¿Mantiene una referencia a ella?	Independiente
Agregación	Todo–parte débil	¿El todo contiene partes que existen sin él?	Independiente
Composición	Todo–parte fuerte	¿Las partes no tienen sentido sin el todo?	Ligado al todo
Generalización	«es un»	¿Es una especialización?	—

Asociación

Dos clases están asociadas cuando una mantiene una **referencia** a la otra. Puede ser unidireccional o bidireccional y tiene una **multiplicidad** (1, 0..1, 0.., *1..*). Ejemplo: un `Curso` tiene un `Profesor` tutor; el profesor existe aunque desaparezca el curso.

- Implementación: atributo de tipo referencia.
- Multiplicidad «muchos» → colección (`List`, `Set`).
- Navegabilidad: ¿quién necesita conocer a quién?

Asociación Curso → Profesor

Java

```java
// Asociación: Profesor y Curso se conocen, pero cada uno existe por sí solo
static class Profesor {
    final String nombre;
    Profesor(String nombre) { this.nombre = nombre; }
}

static class Curso {
    final String titulo;
    Profesor tutor;                      // referencia a otro objeto
    Curso(String titulo, Profesor tutor) { this.titulo = titulo; this.tutor = tutor; }
}
```

Python

```python
# Asociación: Profesor y Curso se conocen, pero cada uno existe por sí solo
class Profesor:
    def __init__(self, nombre):
        self.nombre = nombre


class Curso:
    def __init__(self, titulo, tutor):
        self.titulo = titulo
        self.tutor = tutor            # referencia a otro objeto
```

Agregación

La agregación es una asociación «todo–parte» en la que las partes **pueden existir independientemente** del todo. Un departamento agrupa profesores, pero si el departamento se disuelve los profesores siguen existiendo. En código, el todo recibe las partes ya creadas.

Agregación Departamento — Profesor

Java

```java
// Agregación: el Departamento "tiene" profesores, pero estos viven sin él
static class Departamento {
    final String nombre;
    private final List<Profesor> profesores = new ArrayList<>();
    Departamento(String nombre) { this.nombre = nombre; }
    void incorporar(Profesor p) { profesores.add(p); }     // el objeto llega ya creado
    int total() { return profesores.size(); }
}
```

Python

```python
# Agregación: el Departamento "tiene" profesores, pero estos viven sin él
class Departamento:
    def __init__(self, nombre):
        self.nombre = nombre
        self._profesores = []

    def incorporar(self, profesor):       # el objeto llega ya creado
        self._profesores.append(profesor)

    def total(self):
        return len(self._profesores)
```

Composición

En la composición las partes **no tienen sentido fuera del todo** y su ciclo de vida está ligado a él: el todo las crea y, cuando desaparece, desaparecen con él. Una línea de pedido no existe sin su pedido. Suele implementarse creando las partes dentro del todo y no exponiéndolas directamente.

Composición Pedido — Línea

Java

```java
// Composición: las líneas no tienen sentido fuera del Pedido que las crea
static class Pedido {
    private final List<Linea> lineas = new ArrayList<>();

    void añadir(String producto, int unidades, double precio) {
        lineas.add(new Linea(producto, unidades, precio));  // el Pedido crea y posee
    }

    double total() {
        double t = 0;
        for (Linea l : lineas) t += l.subtotal();
        return t;
    }

    // clase interna: ciclo de vida ligado
    private static class Linea {
        final String producto; final int unidades; final double precio;
        Linea(String p, int u, double pr) { producto = p; unidades = u; precio = pr; }
        double subtotal() { return unidades * precio; }
    }
}
```

Python

```python
# Composición: las líneas no tienen sentido fuera del Pedido que las crea
class Pedido:
    class _Linea:
        def __init__(self, producto, unidades, precio):
            self.producto, self.unidades, self.precio = producto, unidades, precio

        def subtotal(self):
            return self.unidades * self.precio

    def __init__(self):
        self._lineas = []

    def anadir(self, producto, unidades, precio):
        self._lineas.append(Pedido._Linea(producto, unidades, precio))  # el Pedido
        # crea y posee

    def total(self):
        return sum(l.subtotal() for l in self._lineas)
```

> **Truco.** Pregúntate: *si borro el todo, ¿tiene sentido que sobrevivan las partes?* No composición; sí agregación.

Clases contenedoras y objetos colección

Cuando una relación es de «uno a muchos» el objeto guarda una **colección** de referencias. Esta clase contenedora se encarga de añadir, quitar y recorrer los elementos, ocultando la estructura interna. Se usan las colecciones de la biblioteca estándar (`ArrayList`, `HashSet`, `HashMap`) y se devuelve a los clientes una vista de solo lectura.

- Atributo privado `List<Linea>` y métodos `añadir`, `eliminar`, `total`.
- No devolver la lista interna: devolver `List.copyOf(lista)` / `tuple(lista)`.
- El contenedor asume la responsabilidad de mantener la coherencia (p. ej. no duplicados).

Buenas prácticas. Exponer la colección interna rompe la encapsulación: cualquiera podría vaciarla.

Diagrama de clases con relaciones

En UML cada relación tiene su notación: asociación (línea con flecha abierta), agregación (rombo vacío en el lado del todo), composición (rombo relleno), generalización (flecha con triángulo hueco hacia la superclase) y dependencia (línea discontinua).

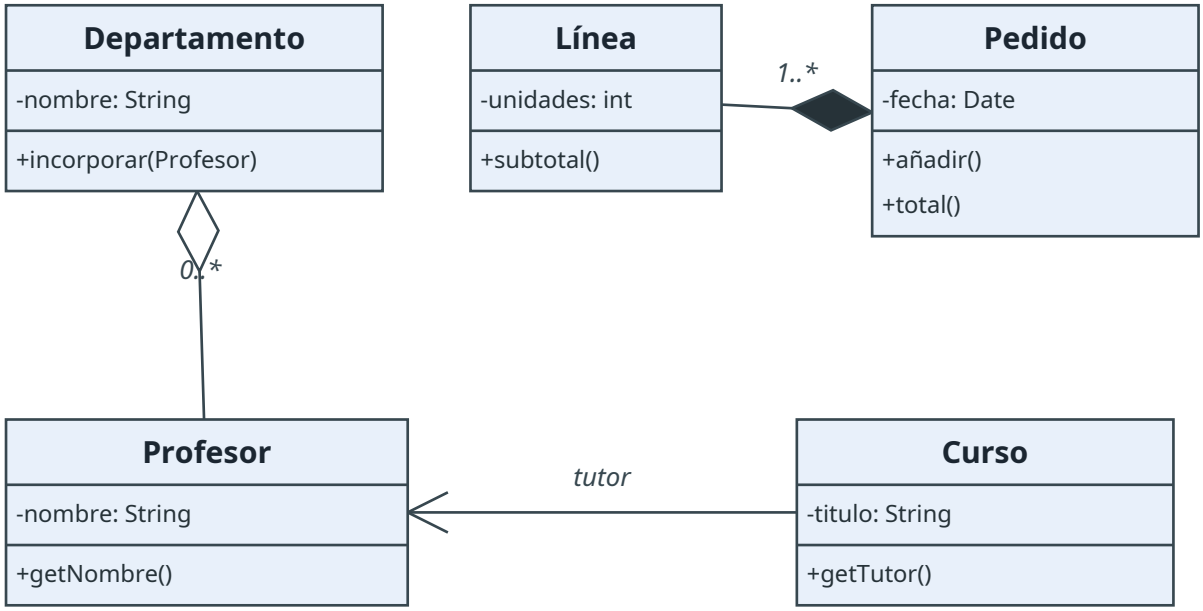


Figura 4.1. Agregación (rombo vacío) Departamento–Profesor, asociación Curso–Profesor y composición (rombo relleno) Pedido–Línea.

Composición frente a herencia

Existe un principio clásico del diseño: «**favorece la composición sobre la herencia**». La herencia acopla fuertemente la subclase a su padre y fija la relación en tiempo de compilación. La composición permite combinar comportamientos y cambiarlos en ejecución.

Criterio	Herencia	Composición
Relación	«es un»	«tiene un» / «usa un»
Acoplamiento	Alto	Bajo
Flexibilidad en ejecución	Baja (fijada al compilar)	Alta (se puede sustituir la parte)
Reutilización	De la implementación del padre	De objetos completos con su interfaz
Cuándo usarla	Especialización real del concepto	Combinar capacidades

Actividad en el aula virtual: Del enunciado al diagrama

Del enunciado al diagrama · 25 minutos

Una tienda online tiene clientes que realizan pedidos. Cada pedido contiene líneas con un producto y una cantidad. Los productos pertenecen a categorías.

- Decidid el tipo de relación entre cada par de clases y justificadlo (10 min).
- Añadid multiplicidades (5 min).
- Contrastad con otro grupo la elección composición/agregación (10 min).

Resumen del tema

- Asociación: referencia entre objetos independientes; agregación: todo–parte débil; composición: todo–parte con ciclo de vida compartido.
- Las relaciones «muchos» se implementan con colecciones encapsuladas.
- Herencia = «es un»; composición = «tiene un». Prefiere composición cuando dudes.
- Los diagramas de clases documentan relaciones y multiplicidades.

Glosario

Término	Significado
Multiplicidad	Número de objetos que pueden participar en una relación (1, 0.., *1*..).
Navegabilidad	Dirección en la que una clase conoce a otra.
Clase contenedora	Clase cuyo cometido es guardar y gestionar una colección de objetos.
Dependencia	Uso puntual de una clase por otra.

Tema 5. Polimorfismo, clases abstractas e interfaces

En este tema. Duración orientativa: 8 horas. Definir polimorfismo y diferenciar sobrecarga, sobreescritura y polimorfismo de subtipos.

Objetivos de aprendizaje

- Definir polimorfismo y diferenciar sobrecarga, sobreescritura y polimorfismo de subtipos.
- Diseñar contratos con clases abstractas e interfaces.
- Programar con referencias al tipo general y enlace dinámico.
- Elegir entre clase abstracta e interfaz.

Criterios de evaluación del módulo que se trabajan

- CE1.7 Polimorfismo y características que introduce en el desarrollo
- CE2.10 Construcción de la interfaz de la clase
- CE2.5 Reconocer las características del paradigma en código

¿Qué es el polimorfismo?

Polimorfismo significa «muchas formas»: un mismo mensaje provoca comportamientos distintos según el tipo real del objeto que lo recibe. Cuando pides «dibuja» a un círculo y a un cuadrado, cada uno sabe cómo hacerlo; quien lo pide no necesita saber de qué figura se trata.

Gracias a él, el código cliente depende de una **abstracción** (la superclase o la interfaz) y no de las clases concretas: se pueden añadir nuevos tipos sin tocar el código existente.

- Un mensaje, muchas respuestas.
- El cliente programa contra el tipo general.
- El tipo real se decide en **ejecución** (enlace dinámico).
- Facilita la extensión sin modificar (principio abierto/cerrado).

Idea clave. Polimorfismo = **sustituibilidad**: donde se espera una `Figura` vale cualquier subclase.

Tipos de polimorfismo

Tipo	Cómo se consigue	Se resuelve en	Ejemplo
Sobrecarga (ad hoc)	Mismo nombre, distinta firma	Compilación	`sumar(int,int)` / `sumar(double,double)`
Sobreescritura (subtipos)	Subclase redefine método heredado	Ejecución	`area()` en `Circulo` y `Rectangulo`
Por interfaz	Clases no relacionadas implementan el mismo contrato	Ejecución	`Comparable`, `Runnable`
Paramétrico (genéricos)	Código válido para varios tipos	Compilación	`List<T>`

> **Aclaración.** Cuando en POO se dice «polimorfismo» a secas se suele hablar del de **subtipos**, el de la fila 2.

Clases abstractas

Una **clase abstracta** representa un concepto demasiado general para ser instanciado («Figura» no se puede dibujar sin saber cuál es). Puede contener atributos, métodos concretos y **métodos abstractos** (sin cuerpo) que obligan a las subclases a implementarlos. No se pueden crear objetos de ella con `new`.

- `abstract class` / `ABC` + `@abstractmethod`.
- Aporta código común y estado compartido.
- Una subclase concreta debe implementar todos los métodos abstractos.

Contrato e implementación base

Java

```java
// Contrato común: define QUÉ se puede hacer, no CÓMO
interface Figura {
    double area();
    default String descripcion() { return getClass().getSimpleName() + " de área " +
        String.format("%.2f", area()); }
}

static abstract class FiguraBase implements Figura {
    protected final String color;
    FiguraBase(String color) { this.color = color; }
    abstract double perimetro();                // método abstracto: lo implementan las
    // hijas
}
```

Python

```python
# Contrato común: define QUÉ se puede hacer, no CÓMO
class Figura(ABC):
    def __init__(self, color):
        self.color = color

    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimetro(self): ...

    def descripcion(self):
        return f"{type(self).__name__} de área {self.area():.2f}"
```

Interfaces

Una **interfaz** es un contrato puro: define qué métodos debe ofrecer una clase, no cómo. Una clase puede implementar **varias** interfaces, lo que sustituye a la herencia múltiple. Desde Java 8 las interfaces admiten métodos `default` y `static`. En Python se emulan con clases abstractas o con `Protocol` (tipado estructural).

Característica	Clase abstracta	Interfaz
Estado (atributos)	Sí	No (solo constantes)

Característica	Clase abstracta	Interfaz
Herencia múltiple	No (una sola)	Sí (varias)
Métodos con código	Sí	Solo `default` / `static`
Relación que expresa	«es un» (jerarquía)	«sabe hacer» (capacidad)
Cuándo usarla	Familia de clases con código común	Contrato común entre clases dispares

Subclases concretas y uso polimórfico

La lista almacena referencias del tipo `Figura`, pero cada elemento apunta a un objeto concreto. Al invocar `descripcion()` o `area()`, el lenguaje elige en ejecución la versión correspondiente al tipo real.

Implementaciones concretas

Java

```java
static class Circulo extends FiguraBase {
    private final double radio;
    Circulo(String color, double radio) { super(color); this.radio = radio; }
    public double area() { return Math.PI * radio * radio; }
    double perimetro() { return 2 * Math.PI * radio; }
}

static class Rectangulo extends FiguraBase {
    private final double base, altura;
    Rectangulo(String color, double base, double altura) { super(color); this.base =
        base; this.altura = altura; }
    public double area() { return base * altura; }
    double perimetro() { return 2 * (base + altura); }
}
```

Python

```python
class Circulo(Figura):
    def __init__(self, color, radio):
        super().__init__(color)
        self.radio = radio

    def area(self):
        return math.pi * self.radio ** 2

    def perimetro(self):
        return 2 * math.pi * self.radio


class Rectangulo(Figura):
    def __init__(self, color, base, altura):
        super().__init__(color)
        self.base, self.altura = base, altura

    def area(self):
        return self.base * self.altura

    def perimetro(self):
        return 2 * (self.base + self.altura)
```

Polimorfismo en acción

Java

```java
// Polimorfismo: una referencia al tipo general, muchos comportamientos
List<Figura> figuras = List.of(new Circulo("rojo", 2), new Rectangulo("azul", 3, 4));
for (Figura f : figuras) {
    System.out.println(f.descripcion());   // la versión ejecutada se decide en
    // ejecución
}
```

Python

```python
figuras = [Circulo("rojo", 2), Rectangulo("azul", 3, 4)]
for f in figuras:
    print(f.descripcion())      # cada objeto responde a su manera
```

Observa. El bucle no sabe qué figura es cada elemento; añadir un `Triangulo` no obliga a modificarlo.

Sobrecarga de métodos

La sobrecarga permite varios métodos con el mismo nombre y distintos parámetros (número o tipo) en una misma clase. El compilador elige cuál llamar según los argumentos. El tipo de retorno **no** distingue una sobrecarga. Python no la soporta por firma: se usan parámetros por defecto, `*args` o `functools.singledispatch`.

Sobrecarga

Java

```java
// Sobrecarga: mismo nombre, distinta firma (se resuelve al compilar)
static double sumar(double a, double b) { return a + b; }
static double sumar(double a, double b, double c) { return a + b + c; }
static String sumar(String a, String b) { return a + b; }
```

Python

```python
# Python no sobrecarga por firma: se usan valores por defecto o *args
def sumar(*valores):
    return sum(valores) if not isinstance(valores[0], str) else "".join(valores)
```

Conversión de tipos y comprobación en ejecución

Subir a la superclase (*upcasting*) es implícito y seguro. Bajar a la subclase (*downcasting*) requiere comprobar el tipo real antes para evitar errores en ejecución: `instanceof` en Java (con *pattern matching* desde Java 16) o `isinstance` en Python. Si necesitas muchos `instanceof`, probablemente falta polimorfismo.

- Upcasting: `Figura f = new Circulo(...)` — siempre válido.
- Downcasting: `(Circulo) f` puede lanzar `ClassCastException`.
- Usa `if (f instanceof Circulo c)` para comprobar y convertir a la vez.
- Muchos `instanceof` encadenados = señal de mal diseño.

Diagrama UML: interfaz y realización

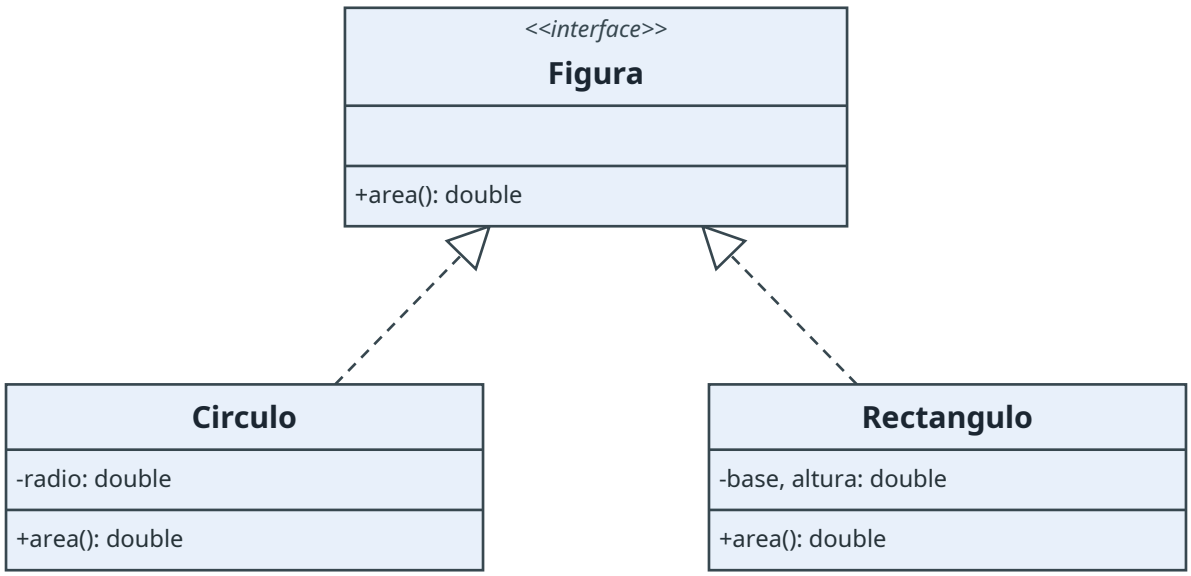


Figura 5.1. Realización de una interfaz (línea discontinua con triángulo hueco).

Actividad en el aula virtual: Añadir una figura sin tocar el bucle

Añadir una figura sin tocar el bucle · 20 minutos

Partiendo del ejemplo de figuras, añadid la clase `Triangulo` y comprobad que el código que recorre la lista no necesita cambios.

- Implementad `Triangulo` (base, altura) (10 min).
- Añadidlo a la lista y ejecutad (5 min).
- Discutid: ¿qué habría pasado con `if/else` por tipo? (5 min).

Resumen del tema

- Polimorfismo: un mensaje, comportamientos distintos según el tipo real.
- Clase abstracta: código común + métodos obligatorios; interfaz: contrato puro.
- Sobrecarga se resuelve al compilar; sobreescritura, al ejecutar.
- Programa contra abstracciones, no contra clases concretas.
- `instanceof` para bajar de tipo con seguridad.

Glosario

Término	Significado
Método abstracto	Método declarado sin implementación; obliga a las subclases a implementarlo.
Interfaz	Contrato de métodos que una clase se compromete a ofrecer.
Enlace dinámico	Selección en ejecución del método según el tipo real del objeto.
Upcasting	Tratar un objeto como su superclase.

Tema 6. Técnicas de programación estructurada

En este tema. Duración orientativa: 10 horas. Escribir algoritmos con secuencia, selección y repetición.

Objetivos de aprendizaje

- Escribir algoritmos con secuencia, selección y repetición.
- Diferenciar funciones y procedimientos y manejar parámetros y valores devueltos.
- Aplicar recursividad y descomposición modular.
- Utilizar las llamadas a métodos dentro de las clases.

Criterios de evaluación del módulo que se trabajan

- CE2.10 Algoritmos con técnicas de programación estructurada y modular
- CE2.6 Entorno de desarrollo asociado al lenguaje

Programación estructurada dentro de la POO

Aunque el programa se organice en objetos, el interior de cada método se escribe con las técnicas de la **programación estructurada**: un flujo claro formado por tres estructuras básicas (secuencia, selección y repetición) y la descomposición en subprogramas. El teorema de Böhm–Jacopini demuestra que con ellas se puede expresar cualquier algoritmo; por eso se evita el `goto`.

- Secuencial: instrucciones una tras otra.
- Condicional (selección): `if / else`, `switch`.
- Repetitiva (iteración): `for`, `while`, `do-while`.
- Modular: dividir en funciones pequeñas con un único propósito.

Variables, tipos y expresiones

Toda variable tiene un **nombre**, un **tipo** y un **ámbito**. Los lenguajes tipados estáticamente (Java) comprueban los tipos al compilar; los dinámicos (Python) al ejecutar. Las expresiones combinan operandos y operadores aritméticos, relacionales y lógicos respetando una precedencia.

Categoría	Operadores	Ejemplo
Aritméticos	`+ - * / %`	`resto = a % b`
Relacionales	`== != < > <= >=`	`edad >= 18`
Lógicos	`&& || !` (Java) / `and or not` (Python)	`esSocio && saldo > 0`
Asignación	`= += -= *=`	`total += precio`

Cuidado. En Java `5 / 2` vale `2` (división entera); usa `5 / 2.0` para obtener `2.5`.

Estructuras de control: selección

La selección permite elegir entre caminos según una condición booleana. `if / else if / else` para condiciones, `switch` para comparar un valor con varios casos. Desde Java 14 el `switch` puede usarse como **expresión** que devuelve un valor; Python incorpora `match / case` desde la versión 3.10.

Funciones: factorial, primos y selección

Java

```java
// Función: devuelve un valor. Procedimiento: no devuelve nada (void)
static int factorial(int n) {                    // recursividad
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

static boolean esPrimo(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {           // estructura de repetición
        if (n % i == 0) return false;
    }
    return true;
}

static void imprimirPrimos(int limite) {          // procedimiento
    StringBuilder sb = new StringBuilder();
    for (int n = 1; n <= limite; n++) {
        if (esPrimo(n)) sb.append(n).append(' ');
    }
    System.out.println("Primos hasta " + limite + ": " + sb);
}
```

Python

```python
# Función: devuelve un valor. Procedimiento: no devuelve nada (None)
def factorial(n):                        # recursividad
    return 1 if n <= 1 else n * factorial(n - 1)


def es_primo(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:                    # estructura de repetición
        if n % i == 0:
            return False
        i += 1
    return True


def imprimir_primos(limite):             # procedimiento
    primos = [str(n) for n in range(1, limite + 1) if es_primo(n)]
    print(f"Primos hasta {limite}:", " ".join(primos))
```

Estructuras de control: repetición

Para cada bucle hay que asegurar tres cosas: **inicialización**, **condición de parada** y **avance** hacia esa condición. Un bucle sin avance es un bucle infinito.

- `for`: número de iteraciones conocido (`for (int i = 0; i < n; i++)`).
- `while`: se repite mientras se cumpla la condición (puede no ejecutarse nunca).
- `do-while`: se ejecuta al menos una vez.

- `for-each`: recorre colecciones sin índices.
- `break` y `continue` alteran el bucle: úsalos con moderación.

> **Error típico.** El error «off-by-one»: recorrer de `1` a `n` en vez de `0` a `n-1`. Lo verás en la UF2406 al depurar.

Funciones y procedimientos

Una **función** devuelve un valor; un **procedimiento** realiza una acción y no devuelve nada (`void` en Java, `None` en Python). En POO a ambos se les llama **métodos** cuando pertenecen a una clase. Un buen método hace una sola cosa, tiene un nombre que expresa su propósito y recibe pocos parámetros.

Elemento	Descripción	Ejemplo Java
Firma	Nombre + tipos de parámetros	`int sumar(int a, int b)`
Parámetros formales	Variables de la declaración	`a`, `b`
Argumentos (reales)	Valores de la llamada	`sumar(3, 4)`
Valor devuelto	Resultado con `return`	`return a + b;`
Ámbito local	Variables visibles solo en el método	`int temp = a * 2;`

Paso de parámetros

Java siempre pasa los argumentos **por valor**: con tipos primitivos se copia el dato; con objetos se copia la **referencia**, de modo que el método puede modificar el objeto apuntado pero no hacer que la variable original apunte a otro. Python funciona de modo parecido (paso «por referencia de objeto»): los tipos inmutables (números, cadenas, tuplas) no pueden cambiarse dentro del método; los mutables (listas, diccionarios) sí.

Paso por valor de primitivos y referencias

Java

```java
// Paso por valor: se copia el valor (primitivos) o la referencia (objetos)
static void modificar(int numero, int[] lista) {
    numero = 99;          // solo cambia la copia local
    lista[0] = 99;        // cambia el objeto compartido
}
```

Python

```python
# Paso por referencia de objeto: los inmutables (int) no cambian; los mutables (list) sí
def modificar(numero, lista):
    numero = 99          # solo cambia la variable local
    lista[0] = 99        # cambia el objeto compartido
```

> **Resultado.** `n` sigue valiendo 1, pero `v[0]` vale 99: se modificó el objeto compartido.

Recursividad

Un método es **recursivo** si se invoca a sí mismo. Necesita un **caso base** que detenga la recursión y un **paso recursivo** que se acerque al caso base. Es natural para estructuras como árboles o para definiciones matemáticas (factorial, Fibonacci), pero consume pila: una recursión muy profunda

provoca `StackOverflowError`.

- Caso base: `factorial(1) = 1`.
- Paso recursivo: `factorial(n) = n * factorial(n-1)`.
- Toda recursión puede reescribirse con un bucle (a veces con una pila explícita).
- Cuidado con la recursión sin caso base o con muchos cálculos repetidos (Fibonacci ingenuo).

Llamadas a métodos dentro de una clase

Dentro de una clase, los métodos se invocan entre sí directamente (`this.metodo()` o simplemente `metodo()`) y pueden compartir el estado del objeto. Conviene dividir un método largo en métodos privados auxiliares: mejora la legibilidad y permite reutilizar y probar cada parte.

- Los métodos `private` auxiliares ocultan el «cómo».
- Método público = parte de la interfaz; privado = detalle de implementación.
- Un método debería caber en una pantalla; si no, dividirlo.
- Nombres con verbos: `calcularTotal`, `esPrimo`, `imprimirResumen`.

Actividad en el aula virtual: Refactorizar un método largo

Refactorizar un método largo · 25 minutos

Se entrega un método de 40 líneas que valida un DNI, calcula la letra e imprime el resultado. Dividirlo en métodos pequeños.

- Identificad las responsabilidades (10 min).
- Extraed métodos con nombres claros (10 min).
- Comparad soluciones y discutid los nombres (5 min).

Resumen del tema

- Secuencia, selección y repetición bastan para cualquier algoritmo.
- Función = devuelve valor; procedimiento = acción sin retorno.
- Java pasa por valor (copia de la referencia para objetos); Python, por referencia de objeto.
- Recursividad = caso base + paso recursivo.
- Dividir en métodos pequeños mejora la calidad.

Glosario

Término	Significado
Ámbito	Zona del programa donde una variable es visible.
Firma	Nombre del método y tipos de sus parámetros.
Caso base	Condición que detiene la recursión.
Procedimiento	Subprograma que ejecuta acciones sin devolver valor.

Tema 7. Estructuras de información y gestión de memoria

En este tema. Duración orientativa: 10 horas. Distinguir los tipos de datos simples y los estructurados.

Objetivos de aprendizaje

- Distinguir los tipos de datos simples y los estructurados.
- Elegir entre arrays, listas, conjuntos, mapas, pilas y colas.
- Implementar una estructura de datos propia como clase.
- Explicar la memoria de pila y de montón, las referencias y la recolección de basura.

Criterios de evaluación del módulo que se trabajan

- CE2.7 Estructuras de datos habituales
- CE2.9 Construcción de clases que representan estructuras de datos
- CE2.2 Gestión de memoria: creación y destrucción de objetos

Tipos de datos simples

Categoría	Java	Python	Notas
Enteros	`byte short int long`	`int` (tamaño ilimitado)	Java: rangos fijos
Reales	`float double`	`float`	Aritmética en coma flotante: errores de redondeo
Lógico	`boolean`	`bool`	`true/false` frente a `True/False`
Carácter	`char`	(cadena de longitud 1)	Unicode
Cadena	`String` (inmutable)	`str` (inmutable)	Objetos, no primitivos
Referencia	Variable de tipo objeto	Toda variable	Apunta a una zona del montón

Importante. `0.1 + 0.2 != 0.3` en coma flotante. Para dinero usa `BigDecimal` (Java) o `Decimal` (Python).

Arrays

Un **array** es una secuencia de elementos del mismo tipo con tamaño fijo y acceso directo por índice (empieza en 0). Es la estructura más sencilla y eficiente, pero no puede crecer. Puede tener varias dimensiones (matrices). Acceder fuera de rango lanza `ArrayIndexOutOfBoundsException`.

Arrays y colecciones

Java

```java
int[] notas = {7, 5, 9, 4};                 // array: tamaño fijo, acceso por índice
int[][] tabla = new int[2][3];              // array bidimensional
tabla[1][2] = 8;
System.out.println("Longitud: " + notas.length + ", suma: " +
    Arrays.stream(notas).sum());
```

Python

```python
notas = [7, 5, 9, 4]                # list: dinámica (para arrays tipados, módulo array
# / numpy)
tabla = [[0] * 3 for _ in range(2)]  # matriz 2x3
tabla[1][2] = 8
print(f"Longitud: {len(notas)}, suma: {sum(notas)}")
```

Colecciones: listas, conjuntos y mapas

La biblioteca estándar ofrece estructuras dinámicas ya implementadas. Elegir la adecuada es una decisión de diseño que afecta al rendimiento y a la claridad.

Estructura	Java	Python	Úsala cuando...
Lista	`ArrayList`, `LinkedList`	`list`	Importa el orden y se repiten elementos
Conjunto	`HashSet`, `TreeSet`	`set`	Sin duplicados; pertenencia rápida
Mapa	`HashMap`, `TreeMap`	`dict`	Buscar por clave
Pila (LIFO)	`ArrayDeque`	`list` (append/pop)	Deshacer, recursión, expresiones
Cola (FIFO)	`ArrayDeque`, `LinkedList`	`collections.deque`	Turnos, tareas pendientes

Código con colecciones

Listas, conjuntos, mapas, pilas y colas

Java

```java
List<String> lista = new ArrayList<>();     // lista dinámica
lista.add("Ana"); lista.add("Luis"); lista.add("Ana");

Set<String> conjunto = new HashSet<>(lista);     // sin duplicados
Map<String, Integer> edades = new HashMap<>();   // pares clave -> valor
edades.put("Ana", 31);
edades.put("Luis", 28);

Deque<Integer> pila = new ArrayDeque<>();        // pila (LIFO)
pila.push(1); pila.push(2);
Queue<Integer> cola = new LinkedList<>();        // cola (FIFO)
cola.offer(1); cola.offer(2);

System.out.println(lista + " " + conjunto.size() + " " + edades.get("Ana"));
System.out.println("pila.pop() = " + pila.pop() + ", cola.poll() = " + cola.poll());
```

Python

```python
lista = ["Ana", "Luis", "Ana"]
conjunto = set(lista)               # sin duplicados
edades = {"Ana": 31, "Luis": 28}    # diccionario clave -> valor
pila = []                           # pila (LIFO)
pila.append(1); pila.append(2)
cola = deque()                      # cola (FIFO)
cola.append(1); cola.append(2)
print(lista, len(conjunto), edades["Ana"])
print(f"pila.pop() = {pila.pop()}, cola.popleft() = {cola.popleft()}")
```

Estructuras no lineales: árboles y grafos

Un **árbol** organiza los datos de forma jerárquica: cada nodo tiene un padre (salvo la raíz) y cero o más hijos; el árbol binario de búsqueda permite buscar en tiempo logarítmico. Un **grafo** es un

conjunto de nodos conectados por aristas (redes de carreteras, relaciones sociales). Se implementan con clases nodo que referencian a otros nodos, o con mapas de adyacencia.

- Árbol: jerarquía (carpetas, organigramas, expresiones).
- Árbol binario de búsqueda: izquierda < nodo < derecha.
- Grafo: nodos + aristas (dirigido o no, con o sin peso).
- En Java: `TreeMap/TreeSet` (árboles rojo-negro) ya listos; los grafos suelen modelarse como `Map<Nodo, List<Nodo>>`.

Implementar una estructura propia

Para entender las estructuras conviene construir una. Una pila con nodos enlazados muestra cómo las **referencias** unen objetos y cómo las operaciones `apilar`, `desapilar` y `cima` se resuelven en tiempo constante. La clase oculta los nodos (composición): el cliente solo ve la interfaz de la pila.

Pila con nodos enlazados (genérica)

Java

```java
class Pila<T> {
    private static class Nodo<T> {                // clase interna: detalle oculto
        final T dato; final Nodo<T> siguiente;
        Nodo(T dato, Nodo<T> siguiente) { this.dato = dato; this.siguiente = siguiente;
            }
    }
    private Nodo<T> cima;
    private int tamaño;

    void apilar(T dato) { cima = new Nodo<>(dato, cima); tamaño++; }
    T desapilar() {
        if (cima == null) throw new IllegalStateException("Pila vacía");
        T dato = cima.dato; cima = cima.siguiente; tamaño--;
        return dato;
    }
    boolean estaVacia() { return cima == null; }
}
```

Python

```python
class Pila:
    class _Nodo:
        def __init__(self, dato, siguiente):
            self.dato, self.siguiente = dato, siguiente

    def __init__(self):
        self._cima, self._tamano = None, 0

    def apilar(self, dato):
        self._cima = Pila._Nodo(dato, self._cima)
        self._tamano += 1

    def desapilar(self):
        if self._cima is None:
            raise IndexError("Pila vacía")
        dato, self._cima = self._cima.dato, self._cima.siguiente
        self._tamano -= 1
        return dato

    def esta_vacia(self):
        return self._cima is None
```

Gestión de memoria: pila y montón

La memoria de un programa Java/Python se organiza en dos zonas principales. La **pila (stack)** guarda las variables locales y las llamadas a métodos; es rápida y se libera sola al salir de cada método. El **montón (heap)** guarda los objetos; allí viven hasta que ya nadie los referencia. Una variable de tipo objeto contiene solo la **referencia** (la dirección) del objeto en el montón.

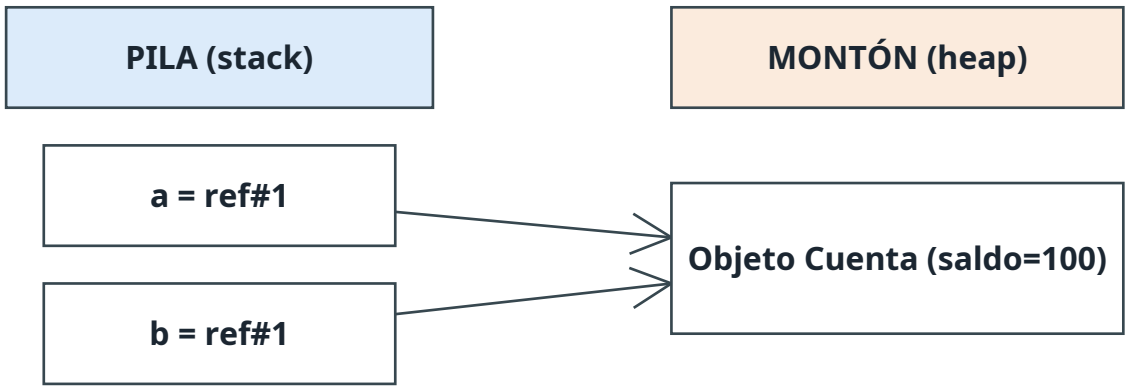


Figura 7.1. Dos variables de la pila apuntan al mismo objeto del montón (copiar la referencia no copia el objeto).

Referencias, copia e igualdad

Referencias frente a copias

Java

```java
// Referencias: dos variables pueden apuntar al MISMO objeto
int[] a = {1, 2, 3};
int[] b = a;                // copia la referencia, no el contenido
int[] c = a.clone();        // copia el contenido
b[0] = 100;
System.out.println("a[0] = " + a[0] + ", c[0] = " + c[0]);

String s1 = new String("hola"), s2 = new String("hola");
System.out.println("== : " + (s1 == s2) + " | equals: " + s1.equals(s2));
```

Python

```python
# Referencias: dos variables pueden apuntar al MISMO objeto
a = [1, 2, 3]
b = a               # copia la referencia
c = a.copy()        # copia el contenido
b[0] = 100
print(f"a[0] = {a[0]}, c[0] = {c[0]}")

s1, s2 = "".join(["ho", "la"]), "".join(["ho", "la"])
print("is :", s1 is s2, "| == :", s1 == s2)
```

Fíjate. `b = a` copia la referencia: cambiar `b[0]` cambia lo que ve `a`. `clone()`/`copy()` crea una copia independiente (superficial).

Recolección de basura y fugas de memoria

El **recolector de basura** (garbage collector) libera automáticamente la memoria de los objetos que no son alcanzables desde ninguna variable activa. No hay `free()`: el programador no libera memoria a mano, pero sí puede provocar **fugas lógicas** manteniendo referencias innecesarias (colecciones estáticas que nunca se vacían, *listeners* sin dar de baja, cachés sin límite).

Python combina **conteo de referencias** con un recolector de ciclos; Java usa recolectores generacionales (G1, ZGC…) que se ejecutan en segundo plano.

- Un objeto es candidato a recolección cuando no es alcanzable.
- `System.gc()` es solo una sugerencia, no una orden.
- Los recursos externos (ficheros, sockets, BD) se cierran explícitamente: `try-with-resources` / `with`.
- Fugas habituales: colecciones estáticas, listeners no eliminados, cachés sin límite.

Actividad en el aula virtual: Elegir la estructura adecuada

Elegir la estructura adecuada · 20 minutos

Para cada necesidad decidid la estructura más adecuada y justificadla: (a) deshacer/rehacer en un editor, (b) cola de impresión, (c) agenda de contactos por nombre, (d) eliminar duplicados de una lista de correos, (e) mapa de metro.

- Decisión individual (5 min).
- Puesta en común con votación en el chat (10 min).
- El docente comenta los casos con más discrepancias (5 min).

Resumen del tema

- Los tipos simples guardan valores; las variables de objeto guardan referencias.
- Array = tamaño fijo; las colecciones (lista, conjunto, mapa, pila, cola) crecen dinámicamente.
- Pila: LIFO; cola: FIFO; mapa: clavevalor; conjunto: sin duplicados.
- Variables locales en la pila; objetos en el montón.
- El recolector de basura libera lo inalcanzable; las fugas lógicas siguen siendo posibles.

Glosario

Término	Significado
Montón (heap)	Zona de memoria donde se crean los objetos.
Pila (stack)	Zona de memoria de variables locales y llamadas a métodos.
LIFO / FIFO	Último en entrar primero en salir / primero en entrar primero en salir.
Referencia	Dirección que identifica un objeto en memoria.
Copia superficial	Copia del objeto que comparte las referencias internas con el original.

Tema 8. Lenguajes orientados a objetos y librerías de clases

En este tema. Duración orientativa: 8 horas. Comparar los lenguajes orientados a objetos más habituales y sus características esenciales.

Objetivos de aprendizaje

- Comparar los lenguajes orientados a objetos más habituales y sus características esenciales.
- Elegir lenguaje y tipo de aplicación según los requisitos.
- Manejar un entorno de desarrollo (IDE) y las herramientas de construcción.
- Localizar y usar las clases básicas de las librerías estándar.

Criterios de evaluación del módulo que se trabajan

- CE2.6 Entorno de desarrollo asociado a un lenguaje OO
- CE2.8 Librerías de clases estándares del lenguaje
- CE1.7-2.5 Características del paradigma en distintos lenguajes

Lenguajes orientados a objetos: panorama

El paradigma orientado a objetos se implementa de formas distintas. Hay lenguajes **puramente** orientados a objetos (Smalltalk, Ruby, donde todo es un objeto), lenguajes **híbridos** que combinan objetos y otros paradigmas (Java, C#, C++, Python, JavaScript, Kotlin) y diferencias importantes en tipado, compilación y gestión de memoria. Conocer sus rasgos permite elegir con criterio.

Lenguaje	Tipado	Ejecución	Memoria	Herencia de clases	Uso típico
Java	Estático	Compilado a bytecode + JVM	Recolector de basura	Simple + interfaces	Empresa, backend, Android
C#	Estático	Compilado a IL + .NET	Recolector de basura	Simple + interfaces	Empresa, escritorio, juegos (Unity)
C++	Estático	Compilado a nativo	Manual (RAII)	Múltiple	Sistemas, juegos, rendimiento
Python	Dinámico	Interpretado (bytecode)	Conteo de refs + GC	Múltiple	Datos, IA, scripting, web
JavaScript / TypeScript	Dinámico / estático opcional	Motor del navegador / Node	Recolector de basura	Prototipos / clases	Web cliente y servidor
PHP	Dinámico (con tipos opcionales)	Interpretado	Recolector de basura	Simple + interfaces	Web servidor

Criterios para elegir lenguaje

- Dominio del problema: web, móvil, datos, sistemas embebidos, escritorio.

- Ecosistema: librerías, frameworks, comunidad y oferta de empleo.
- Rendimiento y consumo de recursos.
- Equipo: conocimientos previos y curva de aprendizaje.
- Mantenimiento: tipado estático, herramientas, soporte a largo plazo.
- Integración con sistemas existentes (BD, servicios, plataformas).

> **En este curso.** Usamos **Java** como lenguaje principal por su tipado estático, la JVM y su peso en el sector, y mostramos **Python** como contraste.

Tipos de aplicaciones

Tipo	Descripción	Tecnologías habituales en Java
Consola	Interacción por texto; scripts y utilidades	`java`, `Scanner`
Escritorio	Interfaz gráfica local	JavaFX, Swing
Web	Cliente ligero + servidor	Servlets, JSP, Spring Boot, Jakarta EE
Móvil	Aplicaciones para dispositivos	Android (Java/Kotlin)
Servicios / API	Funcionalidad expuesta por red	REST con Spring o Jakarta REST
Procesamiento por lotes	Tareas programadas masivas	Spring Batch, `ExecutorService`

Herramientas de desarrollo

El desarrollo profesional se apoya en un conjunto de herramientas integradas. Un **IDE** (entorno de desarrollo integrado) reúne editor con autocompletado, compilador, depurador, ejecución de pruebas y control de versiones. Una **herramienta de construcción** automatiza compilar, probar y empaquetar.

Herramienta	Función	Ejemplos
IDE	Editar, compilar, depurar, refactorizar	IntelliJ IDEA, Eclipse, VS Code, PyCharm
Compilador / intérprete	Traducir y ejecutar el código	`javac`, `java`, `python`
Construcción y dependencias	Compilar, probar, empaquetar, gestionar librerías	Maven, Gradle, pip, venv
Depurador	Ejecutar paso a paso e inspeccionar	jdb, depurador del IDE, pdb
Control de versiones	Historial y trabajo en equipo	Git + GitHub / GitLab
Pruebas	Verificación automática	JUnit, pytest / unittest

Librerías de clases estándar

Gran parte de la productividad en POO viene de **no reinventar**: los lenguajes incluyen miles de clases listas para usar organizadas en paquetes (Java) o módulos (Python). Hay que saber localizar la clase adecuada leyendo la documentación (**Javadoc**, docs.python.org) y entender su contrato.

Área	Java (paquete)	Python (módulo)	Clases / funciones habituales
Núcleo del lenguaje	`java.lang`	(incorporado)	`String`, `Math`, `Object`, `Integer`
Colecciones	`java.util`	`collections`	`List`, `Map`, `Set`, `Deque`
Fechas y tiempo	`java.time`	`datetime`	`LocalDate`, `Duration`
Entrada/salida	`java.io`, `java.nio.file`	`pathlib`, `io`	`Files`, `Path`, `BufferedReader`
Red	`java.net`, `java.net.http`	`urllib`, `http`	`HttpClient`, `URI`
Bases de datos	`java.sql`	`sqlite3`	`Connection`, `PreparedStatement`
Concurrencia	`java.util.concurrent`	`threading`, `concurrent.futures`	`ExecutorService`, `Future`

Cadenas y utilidades básicas

`String` es inmutable: cada operación devuelve una cadena nueva. Para construir texto en un bucle se usa `StringBuilder`. `Math` aporta funciones numéricas.

Cadenas

Java

```java
String texto = "  Programación Orientada a Objetos  ";
System.out.println(texto.trim().toUpperCase());
System.out.println(texto.trim().split(" ").length + " palabras");
StringBuilder sb = new StringBuilder();            // para concatenar en bucles
for (int i = 1; i <= 3; i++) sb.append("UF").append(i).append(' ');
System.out.println(sb.toString().trim());
```

Python

```python
texto = "  Programación Orientada a Objetos  "
print(texto.strip().upper())
print(len(texto.strip().split(" ")), "palabras")
print(" ".join(f"UF{i}" for i in range(1, 4)))   # equivalente a StringBuilder
```

Fechas, streams y Optional

Fechas (java.time / datetime)

Java

```java
LocalDate inicio = LocalDate.of(2026, 10, 26);
LocalDate fin = inicio.plusDays(50);                // curso orientativo
System.out.println("Días entre fechas: " + ChronoUnit.DAYS.between(inicio, fin));
```

Python

```python
inicio = date(2026, 10, 26)
fin = inicio + timedelta(days=50)                  # curso orientativo
print("Días entre fechas:", (fin - inicio).days)
```

Procesar colecciones con streams y evitar null con Optional

Java

```java
List<String> nombres = List.of("Ana", "Luis", "Álvaro", "Beatriz", "Ángel");
List<String> conA = nombres.stream()
        .filter(n -> n.toUpperCase().startsWith("A"))
        .sorted()
        .collect(Collectors.toList());
```

```
System.out.println("Empiezan por A: " + conA);

Optional<String> primero = nombres.stream().filter(n -> n.length() > 6).findFirst();
System.out.println(primero.orElse("ninguno"));
```

Python

```
nombres = ["Ana", "Luis", "Álvaro", "Beatriz", "Ángel"]
con_a = sorted(n for n in nombres if n.upper().startswith("A"))
print("Empiezan por A:", con_a)

primero = next((n for n in nombres if len(n) > 6), None)
print(primero or "ninguno")
```

Idea. Los *streams* expresan **qué** quieres (filtrar, ordenar, recoger) en lugar de **cómo** recorrer con bucles.

Cómo leer la documentación de una API

- Localiza el paquete / módulo y la clase en el índice.
- Lee la descripción de la clase y los **constructores**.
- Revisa los métodos: parámetros, valor devuelto, excepciones (`@throws`).
- Busca el ejemplo de uso y los métodos relacionados («See also»).
- Comprueba la versión: ¿existe en la versión de Java/Python que usas?

Actividad en el aula virtual: Cazadores de clases

Cazadores de clases · 25 minutos

Cada pareja recibe una necesidad (calcular días entre fechas, leer un fichero de texto, ordenar una lista de objetos, hacer una petición HTTP) y debe encontrar en la documentación oficial la clase adecuada y mostrar un ejemplo mínimo.

- Buscar en la documentación (10 min).
- Probar el ejemplo en el IDE (10 min).
- Mostrar al grupo el enlace y el resultado (5 min).

Resumen del tema

- Los lenguajes OO difieren en tipado, ejecución, memoria y herencia múltiple.
- Elegir lenguaje es una decisión técnica y de contexto.
- El IDE, el compilador, la herramienta de construcción, el depurador, Git y las pruebas forman la cadena de herramientas.
- Las librerías estándar evitan reinventar la rueda: aprende a leer su documentación.

Glosario

Término	Significado
IDE	Entorno de desarrollo integrado: editor, compilador, depurador y más.
JVM	Máquina virtual de Java que ejecuta bytecode.

Término	Significado
API	Conjunto de clases y métodos que una biblioteca ofrece al programador.
Javadoc	Documentación HTML generada a partir de los comentarios del código.

Tema 9. Implementación del paradigma: excepciones, eventos e hilos

En este tema. Duración orientativa: 20 horas. Organizar un proyecto Java en paquetes y construir clases completas a partir de un diseño.

Objetivos de aprendizaje

- Organizar un proyecto Java en paquetes y construir clases completas a partir de un diseño.
- Tratar errores con excepciones comprobadas y no comprobadas, y crear excepciones propias.
- Gestionar eventos con el patrón observador y expresiones lambda.
- Programar hilos, sincronizar el acceso a datos compartidos y usar pools de hilos.
- Identificar cómo se accede a bases de datos desde las aplicaciones.

Criterios de evaluación del módulo que se trabajan

- CE2.10 Construcción de clases con relaciones, herencia y tratamiento de errores
- CE2.5 Reconocer las características del paradigma en código
- CE2.6 Entorno de desarrollo
- CE2.10 Acceso a bases de datos desde clases

Del diseño al código: receta para implementar una clase

Hasta ahora hemos visto los conceptos por separado. En este tema los juntamos: dado el diseño detallado de una clase (nombre, atributos, métodos, relaciones, reglas), se construye su código siguiendo siempre los mismos pasos, que garantizan calidad y evitan olvidos.

1. Declarar la clase y su paquete; elegir el nivel de visibilidad.
2. Atributos privados con el tipo (o estructura de datos) más adecuado.
3. Constructores que dejen el objeto en un estado válido (validar).
4. Métodos públicos de la interfaz; métodos privados auxiliares.
5. Relaciones: referencias, colecciones, herencia, interfaces.
6. Tratamiento de errores con excepciones.
7. `toString`, `equals`/`hashCode` si procede.
8. Documentar (Javadoc) y probar.

Estructura de un proyecto Java

Un proyecto se organiza en **paquetes** (carpetas con nombre en minúsculas, normalmente el dominio invertido: `es.academia.poo`). Cada clase pública vive en un fichero con su mismo nombre. El punto de entrada es un método `main`. Para compilar y ejecutar desde la terminal se usan `javac` y `java`; en la práctica lo hace el IDE o Maven/Gradle.

Estructura y ejecución

Texto

```
proyecto/
└── src/main/java/es/academia/poo/
    ├── App.java              ← punto de entrada (main)
    ├── modelo/Cuenta.java    ← entidades
    └── servicio/Banco.java   ← lógica de negocio

$ javac -d out $(find src -name '*.java')
$ java -cp out es.academia.poo.App
```

Convención. Un paquete = un conjunto cohesionado de clases. `import` permite usar clases de otros paquetes sin escribir su nombre completo.

Tipos enumerados y registros

Dos construcciones modernas simplifican mucho el código de datos. Un **enum** define un conjunto cerrado de valores con nombre (estados, tipos, días) y puede tener atributos y métodos. Un **record** (Java 16+) declara una clase inmutable de solo datos: el compilador genera constructor, accesores, `equals`, `hashCode` y `toString`. En Python equivalen a `Enum` y `@dataclass(frozen=True)`.

enum y record

Java

```java
enum Estado { PENDIENTE, EN_CURSO, TERMINADA }

record Tarea(String titulo, Estado estado) {
    Tarea {                                          // constructor compacto: validación
        if (titulo == null || titulo.isBlank()) throw new
            IllegalArgumentException("Título obligatorio");
    }
    Tarea conEstado(Estado nuevo) { return new Tarea(titulo, nuevo); }   // inmutable:
    // devuelve otra
}
```

Python

```python
from dataclasses import dataclass
from enum import Enum


class Estado(Enum):
    PENDIENTE = 1
    EN_CURSO = 2
    TERMINADA = 3


@dataclass(frozen=True)
class Tarea:
    titulo: str
    estado: Estado

    def __post_init__(self):                         # validación
        if not self.titulo.strip():
            raise ValueError("Título obligatorio")

    def con_estado(self, nuevo):                     # inmutable: devuelve otra
        return Tarea(self.titulo, nuevo)
```

Excepciones: concepto y jerarquía

Una **excepción** es un objeto que representa una situación anómala durante la ejecución (fichero inexistente, división por cero, índice fuera de rango). Al lanzarla, el flujo normal se interrumpe y se busca un bloque que la trate subiendo por la pila de llamadas. Separar el tratamiento de errores de la lógica normal hace el código más claro que comprobar códigos de retorno.

Tipo	Descripción	¿Obliga a tratarla?	Ejemplos
Comprobadas (checked)	Condiciones previsibles fuera del control del programa	Sí: `try-catch` o `throws`	`IOException`, `SQLException`, excepciones propias
No comprobadas (runtime)	Errores de programación	No	`NullPointerException`, `IllegalArgumentException`
Errores (Error)	Fallos graves de la JVM	No (no se capturan)	`OutOfMemoryError`, `StackOverflowError`

> **Regla.** Lanza excepciones **significativas**, captura solo lo que sepas resolver y nunca dejes un `catch` vacío.

Excepciones propias y su tratamiento

Una excepción propia hereda de `Exception` (comprobada) o `RuntimeException` (no comprobada) y suele añadir información útil (por ejemplo, cuánto falta). Se captura con `try / catch`; se pueden encadenar varios `catch` de lo más específico a lo más general, y `try-with-resources` cierra los recursos que implementan `AutoCloseable`.

Definir una excepción propia

Java

```java
// Excepción propia (comprobada): obliga a quien llama a tratarla o declararla
static class SaldoInsuficienteException extends Exception {
    private final double faltante;
    SaldoInsuficienteException(double faltante) {
        super("Faltan " + faltante + " EUR");
        this.faltante = faltante;
    }
    double getFaltante() { return faltante; }
}

static class Cuenta {
    private double saldo = 100;
    void retirar(double importe) throws SaldoInsuficienteException {
        if (importe > saldo) throw new SaldoInsuficienteException(importe - saldo);
        saldo -= importe;
    }
}
```

Python

```python
class SaldoInsuficienteError(Exception):
    def __init__(self, faltante):
        super().__init__(f"Faltan {faltante} EUR")
        self.faltante = faltante


class Cuenta:
    def __init__(self):
        self._saldo = 100

    def retirar(self, importe):
```

```
        if importe > self._saldo:
            raise SaldoInsuficienteError(importe - self._saldo)
        self._saldo -= importe
```

try / catch / finally

Java

```java
Cuenta c = new Cuenta();
try {
    c.retirar(30);
    c.retirar(500);                        // lanza la excepción
    System.out.println("No se ejecuta");
} catch (SaldoInsuficienteException e) {
    System.out.println("Operación rechazada: " + e.getMessage());
} finally {
    System.out.println("finally se ejecuta siempre");
}
```

Python

```python
c = Cuenta()
try:
    c.retirar(30)
    c.retirar(500)                    # lanza la excepción
    print("No se ejecuta")
except SaldoInsuficienteError as e:
    print("Operación rechazada:", e)
finally:
    print("finally se ejecuta siempre")
```

> **Orden.** `finally` se ejecuta siempre, haya o no excepción: ideal para liberar recursos.

Liberar recursos automáticamente

Java

```java
// try-with-resources: cierra el recurso automáticamente
static class Conexion implements AutoCloseable {
    Conexion() { System.out.println("Conexión abierta"); }
    void consultar() { System.out.println("Consultando..."); }
    @Override public void close() { System.out.println("Conexión cerrada"); }
}
```

Python

```python
# Gestor de contexto: equivalente a try-with-resources
class Conexion:
    def __enter__(self):
        print("Conexión abierta")
        return self

    def consultar(self):
        print("Consultando...")

    def __exit__(self, tipo, valor, traza):
        print("Conexión cerrada")
        return False
```

Gestión de eventos: el patrón observador

En una aplicación interactiva o reactiva el flujo no lo marca el programa sino lo que ocurre: un clic, un pedido que cambia de estado, un mensaje que llega. Un **evento** es un objeto que describe lo ocurrido; un **oyente (listener)** es el código que se ejecuta como respuesta; la **fuente** notifica a

todos sus oyentes. Es el patrón **Observador**, que desacopla al que emite del que reacciona.

- Fuente del evento: el objeto que cambia (botón, pedido, sensor).
- Evento: información del cambio (`EventoPedido`).
- Oyente: interfaz con un método que reacciona (`alCambiarEstado`).
- Registro: la fuente guarda la lista de oyentes y los notifica.
- Las expresiones lambda permiten registrar el oyente en una línea.

Eventos en código

Fuente, evento y oyente

Java

```java
// Patrón Observador: la base de la gestión de eventos (GUI, web, sensores...)
record EventoPedido(String codigo, String estado) { }

@FunctionalInterface
interface PedidoListener {
    void alCambiarEstado(EventoPedido evento);
}

static class Pedido {
    private final String codigo;
    private final List<PedidoListener> oyentes = new ArrayList<>();

    Pedido(String codigo) { this.codigo = codigo; }

    void addListener(PedidoListener l) { oyentes.add(l); }

    void cambiarEstado(String nuevo) {
        EventoPedido e = new EventoPedido(codigo, nuevo);
        oyentes.forEach(l -> l.alCambiarEstado(e));   // notificación a todos los
        // suscriptores
    }
}
```

Python

```python
# Patrón Observador: la base de la gestión de eventos (GUI, web, sensores...)
@dataclass(frozen=True)
class EventoPedido:
    codigo: str
    estado: str


class Pedido:
    def __init__(self, codigo):
        self._codigo = codigo
        self._oyentes = []

    def add_listener(self, listener):      # cualquier función que reciba el evento
        self._oyentes.append(listener)

    def cambiar_estado(self, nuevo):
        evento = EventoPedido(self._codigo, nuevo)
        for oyente in self._oyentes:       # notificación a todos los suscriptores
            oyente(evento)
```

Suscripción de oyentes

Java

```java
Pedido p = new Pedido("P-001");
p.addListener(e -> System.out.println("[Log] " + e.codigo() + " -> " + e.estado()));
p.addListener(e -> {
    if (e.estado().equals("ENVIADO")) System.out.println("[Email] Aviso al cliente de "
        + e.codigo());
});
p.cambiarEstado("PREPARADO");
p.cambiarEstado("ENVIADO");
```

Python

```python
p = Pedido("P-001")
p.add_listener(lambda e: print(f"[Log] {e.codigo} -> {e.estado}"))
p.add_listener(lambda e: print(f"[Email] Aviso al cliente de {e.codigo}") if e.estado
    == "ENVIADO" else None)
p.cambiar_estado("PREPARADO")
p.cambiar_estado("ENVIADO")
```

Ventaja. Se pueden añadir nuevos oyentes (auditoría, avisos, métricas) sin modificar `Pedido`.

Hilos: ejecución concurrente

Un **hilo** (thread) es un flujo de ejecución independiente dentro de un proceso; varios hilos comparten la memoria del proceso. Permiten aprovechar varios núcleos y mantener la aplicación receptiva mientras se hace una tarea lenta (descargar, calcular, consultar una BD). El precio es la complejidad: **condiciones de carrera**, **interbloqueos** y errores difíciles de reproducir.

Estado del hilo	Significado
Nuevo	Creado pero no iniciado (`new Thread(...)`)
Ejecutable	Listo para ejecutarse (`start()`)
En ejecución	La CPU lo está ejecutando
Bloqueado / en espera	Esperando un recurso, un `join()`, un `sleep()` o un monitor
Terminado	Su método `run()` ha finalizado

Crear y sincronizar hilos

Crear y esperar a un hilo

Java

```java
Thread t = new Thread(() -> System.out.println("Hola desde " +
    Thread.currentThread().getName()), "hilo-1");
t.start();
t.join();
```

Python

```python
t = threading.Thread(target=lambda: print("Hola desde",
    threading.current_thread().name), name="hilo-1")
t.start()
t.join()
```

Acceso seguro a datos compartidos

Java

```java
// Sin sincronización, varios hilos pueden pisarse al modificar el mismo dato
static class ContadorInseguro {
    int valor = 0;
```

```
    void incrementar() { valor++; }              // no es atómico: leer-sumar-escribir
}

static class ContadorSeguro {
    private int valor = 0;
    synchronized void incrementar() { valor++; } // solo un hilo a la vez
    synchronized int getValor() { return valor; }
}
```

Python

```
# Sin sincronización, varios hilos pueden pisarse al modificar el mismo dato
class ContadorSeguro:
    def __init__(self):
        self._valor = 0
        self._lock = threading.Lock()

    def incrementar(self):
        with self._lock:                # solo un hilo a la vez
            self._valor += 1

    @property
    def valor(self):
        return self._valor
```

Condición de carrera. `valor++` son tres pasos (leer, sumar, escribir): sin `synchronized` / `Lock` dos hilos pueden pisarse y perder incrementos.

Pools de hilos y resultados futuros

Java

```
// En la práctica se usan pools de hilos en lugar de crear hilos a mano
ExecutorService pool = Executors.newFixedThreadPool(3);
AtomicInteger suma = new AtomicInteger();
Future<Integer> resultado = pool.submit(() -> {
    for (int i = 1; i <= 10; i++) suma.addAndGet(i);
    return suma.get();
});
System.out.println("Resultado asíncrono: " + resultado.get());
pool.shutdown();
```

Python

```
# En la práctica se usan pools de hilos en lugar de crear hilos a mano
with ThreadPoolExecutor(max_workers=3) as pool:
    resultado = pool.submit(lambda: sum(range(1, 11)))
    print("Resultado asíncrono:", resultado.result())
```

Buenas prácticas con hilos

- Prefiere **pools** (`ExecutorService`) a crear hilos manualmente.
- Minimiza el estado compartido; mejor datos inmutables o locales al hilo.
- Usa clases concurrentes (`AtomicInteger`, `ConcurrentHashMap`) antes que `synchronized` manual.
- Evita interbloqueos: adquirir los bloqueos siempre en el mismo orden.
- Python: el GIL limita el paralelismo de CPU en hilos; para cálculo intensivo usa `multiprocessing`.

Cuidado. La interfaz gráfica y la mayoría de los frameworks solo permiten tocar la interfaz desde un hilo concreto.

Acceso a bases de datos desde las aplicaciones

Casi toda aplicación real guarda sus datos en una base de datos relacional. Desde una aplicación OO se accede mediante una API estándar —**JDBC** en Java, **DB-API** en Python— que ofrece clases para abrir la conexión, ejecutar sentencias SQL y recorrer los resultados. En la UF2405 profundizaremos; aquí basta con conocer el esquema.

- `DriverManager.getConnection(url, usuario, clave)` `Connection`.
- `PreparedStatement` para consultas parametrizadas (nunca concatenar datos del usuario).
- `ResultSet` para recorrer las filas devueltas por un `SELECT`.
- Las clases del dominio no deben contener SQL: se aísla en un DAO / repositorio.
- Siempre cerrar `Connection`, `Statement` y `ResultSet` (`try-with-resources`).

Proyecto integrador de la UF2404

Para consolidar la unidad, el alumnado desarrolla un pequeño sistema orientado a objetos de principio a fin: análisis, diagrama de clases, implementación con herencia, interfaces, colecciones y excepciones, y una demostración en el aula virtual.

- Tema a elegir: gestión de biblioteca, tienda, gimnasio, reservas…
- Mínimo: 6 clases, una jerarquía de herencia, una interfaz, una relación de composición y excepciones propias.
- Entrega: código (repositorio), diagrama UML, breve informe y vídeo de 5 minutos.
- Evaluación con la rúbrica de la guía didáctica.

Actividad en el aula virtual: Reaccionar a eventos con hilos

Reaccionar a eventos con hilos · 40 minutos

Ampliad el ejemplo de `Pedido`: cada oyente debe ejecutarse en un hilo del pool. Comprobad que el orden de las salidas deja de ser determinista y arreglad un contador de notificaciones compartido.

- Pasad los oyentes a un `ExecutorService` (15 min).
- Provocad una condición de carrera con un contador (10 min).
- Corregidla con `AtomicInteger` o `synchronized` (15 min).

Resumen del tema

- Implementar una clase desde un diseño sigue siempre los mismos pasos.
- Las excepciones separan el flujo normal del de error; las propias aportan significado.
- Eventos = fuente + evento + oyentes (observador); desacoplan emisor y receptor.
- Los hilos dan concurrencia pero exigen sincronizar el estado compartido.
- El acceso a BD se hace con JDBC/DB-API y se aísla en DAOs.

Glosario

Término	Significado
Excepción	Objeto que representa una situación anómala durante la ejecución.

Término	Significado
Listener	Objeto que reacciona a un evento.
Condición de carrera	Resultado incorrecto por depender del orden de ejecución de varios hilos.
Pool de hilos	Conjunto reutilizable de hilos que ejecutan tareas.
JDBC	API estándar de Java para acceder a bases de datos relacionales.